

Generative machine learning approaches to optimization

Victor Alves and John R. Kitchen

December 17, 2025

Contents

1	Introduction	2
2	Methods	4
2.1	Gaussian mixture models	4
2.1.1	Training	5
2.1.2	Prediction	5
2.2	Utility functions	6
3	Results and discussion	8
3.1	Root finding	8
3.1.1	Finding imaginary roots	12
3.2	Optimization with generative models	26
3.3	Optimization with an equality constraint	30
3.4	Constrained optimization with an inequality	33
3.5	Parameter estimation with generative models	36
3.5.1	Caveats	41
3.6	Mapping input and output spaces	42
4	Conclusions	46
5	Acknowledgments	46

Abstract

Solving optimization problems, especially for nonlinear and constrained systems, is a challenge. Decades of specialized algorithms have been developed for general and special cases of root finding, minimization (including constraints), for parameter estimation, and mapping connected spaces. These approaches typically require a model, or set of equations, and then analytical, or iterative numerical approaches can be used to find a solution, and in some special cases a globally best solution can be found and proven. In this work we present an alternative approach to solving these problems that is based in generative machine learning models. The idea is these models either estimate a joint probability distribution of input and output variables, or are able to transform a distribution of inputs to a distribution of outputs (or vice versa). Then, by suitable conditioning (specifying desired properties of some variables), the solutions to these problems can be obtained by sampling the distribution, or by transformation of a distribution sample to the solution space. We illustrate the approach with Gaussian mixture models, which approximate the joint probability distribution. We show examples in root finding, unconstrained and constrained optimization, parameter estimation and mapping input and output spaces. This work is intended to be pedagogical to show how a generative model

can be used to solve problems in optimization. We discuss some limitations of this approach, but conclude the approach has promise and is different than existing approaches.

1 Introduction

The field of optimization is immense, with elegant solutions tailored to specific problem applications, depending on the nature of the objective function and constraints that are to be solved (e.g., linear *vs.* nonlinear, convex *vs.* nonconvex, differentiable *vs.* nondifferentiable), the nature of the variables involved in the optimization problem (discrete, continuous, or both), and lastly, the nature of the solution itself, global *vs.* local optimization. This myriad of problems that arise in many fields of science and engineering gave birth to several solution techniques, ranging from the seminal work of linear programming and the Simplex method by Dantzig,¹ to the advent of robust nonlinear programming techniques including sequential quadratic programming (SQP),²⁻⁴ and the popularization of interior-point methods.⁵ Additionally, solutions of mixed integer nonlinear programming (MINLP) algorithms⁶ and recent innovations on generalized disjunctive programming (GDP) which embeds logical decisions into mathematical programming formulations,⁷ and global optimization⁸⁻¹⁰ are all examples of how rich is the literature on the topic. In Python the `scipy` library is particularly notable,¹¹ but others also exist like `Pyomo`,¹² `GEKKO`,¹³ `Gurobi`,¹⁴ `Mosek`,¹⁵ and `JuMP`.¹⁶ These libraries include both algebraic optimization modeling languages as well as commercial and/or open-source optimization solvers. Despite all the work aforementioned, the list is by no means exhaustive, and many other interesting research directions are currently available and being actively developed.

We see a particular interest in algorithmic development of solutions to optimization problems that are specific to the nature of the problem, its variables and local/optimal convergence criteria. However, less attention has been given to the use of emerging and powerful tools such as generative machine learning (genML) in the well-developed area of optimization. It is clear that optimization/mathematical programming is one of the cornerstones of machine learning (independent of the generative nature or lack thereof), but the reverse application, that is, genML as a tool to solve optimization problems, seems to be largely unexplored.

To orient the reader on where a generative-based optimization approach fits in the optimization landscape, we illustrate genML-opt in Figure 1, based on the classification of optimization problems in Ref. 17. We intentionally modified the original flowchart in Refs. 17,18 to show which class of problems we are able to currently employ genML-opt. Although promising, there is still a lot to cover when compared to the current state-of-the-art in optimization.

In this manuscript we explore a different way to think about optimization through generative machine learning (genML).¹⁹ The basic concept in genML is that if one knows the joint probability distribution of a set of variables, then it is possible to generate samples of that distribution. Alternatively, if one knows how to transform a reference distribution to the joint probability distribution of those variables, then one samples the reference distribution and transforms it to a sample of the desired distribution.

In fact, generative models have been applied to optimization problems before. Invertible neural networks are a special class of models that are used to model the joint distribution of inputs and outputs.²⁰ Once they are trained, feasible solutions can be found by sampling them. Diffusion models have been applied to inverse problems in mechanics.²¹ We recently applied genML to solve an inverse linear problem.²² Generative models can also be used to enhance combinatorial optimization problems²³ where they can be used to generate near-optimal solutions that are subsequently refined by conventional algorithms. Finally, generative adversarial models have been used

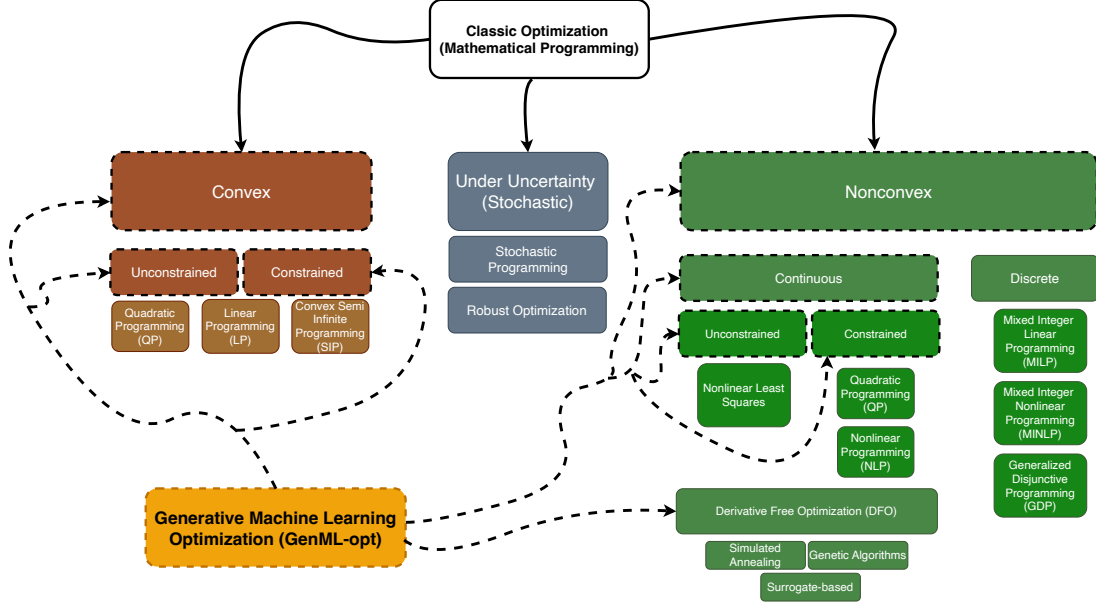


Figure 1: Tree containing classes of optimization problems, based on flowcharts available in the literature.^{17,18} We show with connected dashed lines and also highlighting classes of optimization problems that we were able to currently solve using generative machine learning applied to optimization - GenML-opt.

in optimization²⁴

Probably the most common genML method used today is in the generation of text²⁵ and images.²⁶ Large language models (LLMs) are a form of genML where the most likely sequence of words to follow a prompt is generated. LLMs are not the focus of this work. In image generation, a genML model is trained on a large set of images to "learn the image distribution", and then images are generated by sampling that distribution. We find it helpful to think of an image first as a matrix of pixel values (e.g. the RGB intensities), and then to think of that matrix as a flattened vector. Then, a large set of images will have a distribution of values in that vector space where there are correlations (covariance) between the elements of the vector that define an image space. Thus, one generates a sample vector, and then reshapes it into a matrix, which can be visualized as an image. This is the motivation for the present work.

That does not sound immediately helpful in solving optimization problems until we note two things. First, instead of a collection of pixel values, the vector elements may be inputs and outputs of a model or system, and by concatenating those we obtain a vector analogous to the flattened image vector. The vector elements may even include derivatives of the model, or other easily derived features. If we have a large number of these vectors, then they will form a distribution in that vector space where there are correlations (covariance) between the elements, e.g. the inputs and outputs. Since it is evident generative models can learn that distribution for images, it seems reasonable they could learn this distribution for inputs/outputs, which we demonstrate in this work.

Thus, if we can learn or estimate the joint probability distribution, we can generate samples of inputs and outputs from it. Second, we must introduce the idea of conditional generation, that is, we can specify some values of the variables, update the joint probability distribution of the remaining variables, and then generate those variables consistent with our condition. Thus, if we condition on inputs, we can then generate samples of the output, and vice versa. This is directly

related to root finding; we might condition on the output being zero, and generate the inputs that lead to that result.

To connect this idea to finding minima/maxima we simply need to see the connection between inputs of a model and its derivatives. If we know that joint probability distribution, then we generate samples conditioned on the derivatives being equal to zero. We can include constraints by leveraging a Lagrangian function, or another augmented function and condition the derivatives of that to be zero. Finally, in parameter estimation, we need the joint probability distribution between the parameters and an observed output(s). Then to estimate parameters from an observation, we simply condition on the observation to find the parameters. These are all related to solving inverse problems in optimization.

We think this approach is fundamentally different than a conventional view point. There may be a model involved, but it is not necessary; the approach can be completely data-driven in some cases. We do not need to think about a forward or inverse model. The joint probability distribution (or the transformation function) is simultaneously both of these; conditioning determines what direction if any one thinks in (for example, it is possible to condition on some inputs and outputs). That means we get solutions to optimization problems without using conventional approaches like Newton-Raphson iterative methods, or other specialized algorithms like simplex methods to solve nonlinear programs. We make no claims of superiority of generative methods over the well-established methods; indeed, conventional algorithms offer many advantages to the ideas presented here. Instead, we focus on developing a proof of concept for a new way to think about solving these problems. We do this through a series of examples that span root finding, minimization including constraints, parameter estimation, and state mapping.

GenML has the potential to serve as an alternative optimization solution approach to several classes of problems listed above, including linear and nonlinear programming (both constrained and unconstrained), parameter estimation problems, inverse mapping, and root finding, to name a few. Intriguingly, we have observed that the solution approach to these types of problems is remarkably consistent, as opposed to highly-tailored and specific algorithms (the current *status quo*) as long as data is available. We do not anticipate that generative machine learning formulations will replace classic mathematical programming development. Instead, we believe that a branch of optimization problems that can be solved by using GenML will employ even more sophisticated optimization algorithms at the training and inference of generative models. In short, the optimization layer will always be present, and its interaction with the researcher/practitioner will be what might change within the next years.

2 Methods

We will focus on Gaussian Mixture Models (GMM) in this work, particularly an implementation in `gmr`.²⁷ There are other generative models based on diffusion and continuous flow matching²⁸ that also work, but for the purpose of this paper, they function practically the same. Analogous to the "no free lunch" theorem, there is no one-size-fits all generative model; but, with enough data and training, they all perform the same on average.²⁹ This is analogous to it not mattering whether one chooses a neural network or Gaussian process model; with enough data they are both useful, and the choice is often made out of convenience or preference.

2.1 Gaussian mixture models

Gaussian Mixture Models (GMMs) are a powerful and versatile mathematical tool that is able to characterize complex and multimodal distributions by combining multiple individual Gaussians.

They are used in machine learning applications related but not limited to data clustering.^{30,31} They are also recognized in the literature related to pattern recognition in machine learning.³² Furthermore, in the field of robotics, Gaussian Mixture Regression (GMR) has been used as a generative modeling framework,³³ and it is considered a generative model.³⁴ We follow the notation from Ref 33 for the mathematical definition of GMMs. However, the reader should note that equally valid formulations are present in the literature.^{32,34}

From a machine learning perspective, the main task is, given a pair of vectors x and y , to learn a probability distribution as defined as in Eq. 1, which is the definition of a Gaussian mixture model.

$$p(\mathbf{x}, \mathbf{y}) = \sum_{k=1}^K \pi_k \mathcal{N}_k(\mathbf{x}, \mathbf{y} \mid \boldsymbol{\mu}_{\mathbf{x}\mathbf{y}_k}, \boldsymbol{\Sigma}_{\mathbf{x}\mathbf{y}_k}) \quad (1)$$

In which $\mathcal{N}_k(\mathbf{x}, \mathbf{y} \mid \boldsymbol{\mu}_{\mathbf{x}\mathbf{y}_k}, \boldsymbol{\Sigma}_{\mathbf{x}\mathbf{y}_k})$ are the k^{th} Gaussian distributions in a mixture, which are defined by their respective means $\boldsymbol{\mu}_{\mathbf{x}\mathbf{y}_k}$ and covariances $\boldsymbol{\Sigma}_{\mathbf{x}\mathbf{y}_k}$. $\pi_k \in [0, 1]$ are defined as *weights* or *priors*, which quantify the contribution of each Gaussian to the mixture model.

2.1.1 Training

The training step corresponds to obtaining an accurate prediction of this unknown, probably multimodal and multivariable probability distribution, given a dataset defined by the pair of vectors x, y . For simplicity, we can lump the parameters related to the formulation of Eq. 1 which includes the means, covariances and mixture weights as $\boldsymbol{\theta} := \{\pi_k, \boldsymbol{\mu}_{\mathbf{x}\mathbf{y}_k}, \boldsymbol{\Sigma}_{\mathbf{x}\mathbf{y}_k} : k = 1, \dots, K\}$

As a figure of merit is needed to fit this generative machine learning model, we resort to a likelihood formulation, defined as in Eq.2

$$p(x, y \mid \boldsymbol{\theta}) = \prod_{n=1}^N p(x_n, y_n \mid \boldsymbol{\theta}), \quad p(x_n, y_n \mid \boldsymbol{\theta}) = \sum_{k=1}^K \pi_k \mathcal{N}(x, y \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k) \quad (2)$$

The log-likelihood³⁴ is formulated in Eq.3. Ideally, one could solve for $d\mathcal{L}/d\boldsymbol{\theta} = 0$ and the optimal parameters $\boldsymbol{\theta}^*$ that maximizes the likelihood would be obtained analytically. However, since Eq. 3 is not a closed-form expression/solution,³⁴ an iterative method is used. Expectation Maximization (EM) is applied in GMMs successfully and this is what packages such as `gmr`³³ and the `scikit-learn`³⁵ implementation of GMM does.

$$\log p(\mathbf{x}, \mathbf{y} \mid \boldsymbol{\theta}) = \sum_{n=1}^N \log p(\mathbf{x}_n \mid \boldsymbol{\theta}) = \underbrace{\sum_{n=1}^N \log \sum_{k=1}^K \pi_k \mathcal{N}(\mathbf{x}_n \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)}_{=: \mathcal{L}} \quad (3)$$

2.1.2 Prediction

Following the conditional distribution property of each individual Gaussian, one is able to compute the conditional distribution $p(y \mid x, \boldsymbol{\theta})$ by first writing down the conditional distribution for a given k^{th} element.³³

$$\boldsymbol{\mu}_{\mathbf{x}\mathbf{y}_k} = \begin{bmatrix} \boldsymbol{\mu}_{\mathbf{x}k} \\ \boldsymbol{\mu}_{\mathbf{y}k} \end{bmatrix}, \quad \boldsymbol{\Sigma}_{\mathbf{x}\mathbf{y}_k} = \begin{bmatrix} \boldsymbol{\Sigma}_{\mathbf{x}\mathbf{x}_k} & \boldsymbol{\Sigma}_{\mathbf{x}\mathbf{y}_k} \\ \boldsymbol{\Sigma}_{\mathbf{y}\mathbf{x}_k} & \boldsymbol{\Sigma}_{\mathbf{y}\mathbf{y}_k} \end{bmatrix} \quad (4)$$

The conditional distribution of $\mathbf{y}$ given $\mathbf{x}$ for the k^{th} Gaussian component is defined as in Eq.5:

$$\begin{aligned}\boldsymbol{\mu}_{y|x}^{(k)} &= \boldsymbol{\mu}_{y_k} + \boldsymbol{\Sigma}_{yx_k} \boldsymbol{\Sigma}_{xx_k}^{-1} (\mathbf{x} - \boldsymbol{\mu}_{x_k}) \\ \boldsymbol{\Sigma}_{y|x}^{(k)} &= \boldsymbol{\Sigma}_{yy_k} - \boldsymbol{\Sigma}_{yx_k} \boldsymbol{\Sigma}_{xx_k}^{-1} \boldsymbol{\Sigma}_{xy_k}\end{aligned}\tag{5}$$

To obtain the full conditional distribution $p(\mathbf{y} | \mathbf{x})$, we combine the component-wise conditional Gaussians using the posterior weights $\pi_{k|x}$, which represent the probability of component k given the observed $\mathbf{x}$:

$$\pi_{k|x} = \frac{\pi_k \mathcal{N}(\mathbf{x} | \boldsymbol{\mu}_{x_k}, \boldsymbol{\Sigma}_{xx_k})}{\sum_{l=1}^K \pi_l \mathcal{N}(\mathbf{x} | \boldsymbol{\mu}_{x_l}, \boldsymbol{\Sigma}_{xx_l})}\tag{6}$$

Finally, the final predicted distribution corresponds to a mixture of conditional Gaussians:

$$p(\mathbf{y} | \mathbf{x}) = \sum_{k=1}^K \pi_{k|x} \mathcal{N}(\mathbf{y} | \boldsymbol{\mu}_{y|x}^{(k)}, \boldsymbol{\Sigma}_{y|x}^{(k)})\tag{7}$$

If one desires to have them in a more compact form, we refer to our previous definition of the lumped hyperparameters as θ as

$$\theta =: \left\{ \pi_k, \boldsymbol{\mu}_{xy_k}, \boldsymbol{\Sigma}_{xy_k} \right\}_{k=1}^K\tag{8}$$

which yields a predictive distribution form as in Eq. 9.

$$p(\mathbf{y} | \mathbf{x}; \theta) = \sum_{k=1}^K \pi_{k|x}(\theta) \mathcal{N}(\mathbf{y} | \boldsymbol{\mu}_{y|x}^{(k)}, \boldsymbol{\Sigma}_{y|x}^{(k)})\tag{9}$$

Eq. 9 plays a major role in our work. As we draw samples based on conditioning, we can simultaneously infer in variables that belong to both the x or y spaces. We show this in several applications related to optimization.

Lastly, we note that typical optimization techniques are used, more specifically during the training step of the generative approach, namely the EM algorithm.³⁶ However, this optimization step that requires $\mathcal{O}(\mathcal{D}^3)$ for a dataset of dimensionality $\mathcal{D}$, is performed offline and separately from the applications in which the GM model is employed.

2.2 Utility functions

We do have to do some hyperparameter tuning with a GMM, most notably choosing the number of components in the model. To avoid a lot of duplicated code in this work, we develop this function to automate that choice for us based on an information criteria. This function uses Bayesian Information Criteria (BIC) to pick the best GMM for a given data set.

```
from sklearn.mixture import GaussianMixture
import numpy as np
from gmr import GMM

def best_gmm_bic(X, maxN=None):
    """
    Find the best Gaussian Mixture Model (GMM) for the given data.

    This function uses the Bayesian Information Criterion (BIC) to evaluate
    the fit of Gaussian Mixture Models with varying number of components, and
    returns the model with the lowest BIC value.
```

```

Parameters
-----
X : array-like, shape (n_samples, n_features)
    The input data to be modeled by the GMM.

maxN : int, optional
    The maximum number of components to consider for the GMM.
    If None, a default value will be used.

Returns
-----
best_gmm : GaussianMixture
    The Gaussian Mixture Model with the lowest BIC for the given data.
"""

bics = []
models = []

for k in range(1, min(100, maxN or len(X))):
    model = GaussianMixture(n_components=k,
                            covariance_type='full',
                            random_state=0).fit(X)

    gmm = GMM(n_components=model.n_components,
              priors=model.weights_,
              means=model.means_,
              covariances=model.covariances_)

    bics.append(model.bic(X))
    models.append(gmm)

best_k = np.argmin(bics)
model = models[best_k]

model.models = models
model.bics = bics

return model

```

We will also need a convenient way to analyze clusters of samples. We use a density-based clustering algorithm to identify the clusters, and then do statistical analysis on those clusters.

```

from sklearn.cluster import DBSCAN

def cluster_stats(X):
    """Compute mean and stdev for clusters in an array.

    This function is used for the side-effect of printing those results.

    Parameters
    -----
    X : array-like

    Returns
    -----
    None

    """
    X = np.atleast_2d(X)
    clustering = DBSCAN().fit(X)
    labels = clustering.labels_

    for L in set(labels):
        mean = np.mean(X[labels == L])
        std = np.std(X[labels == L])

```

```
n = len(X[labels == L])
print(f'{L:3d}: {mean: 5.3f} +\\- {std:1.4f} (n={n})')
```

It is convenient to set some default parameters for figures here. These settings make the figures transparent and higher resolution.

```
import matplotlib as mpl

mpl.rcParams['figure.facecolor'] = 'none'
mpl.rcParams['axes.facecolor'] = 'none'
mpl.rcParams['figure.dpi'] = 300 # Set DPI to 300
```

3 Results and discussion

3.1 Root finding

In root finding we seek to find the values of x where $f(x) = 0$. We demonstrate this first on a polynomial where there are two real and two imaginary roots. We start by creating data that represent samples of x and the corresponding values of y . This data is sampled from a function, but it could also be observation data from an instrument, for example. Here we consider a polynomial $y = x^4 - 4x^2 + x + 3$. This function has easily computable roots for comparison with the roots we subsequently obtain by a generative model. We find those roots next with a conventional function `numpy.roots`.

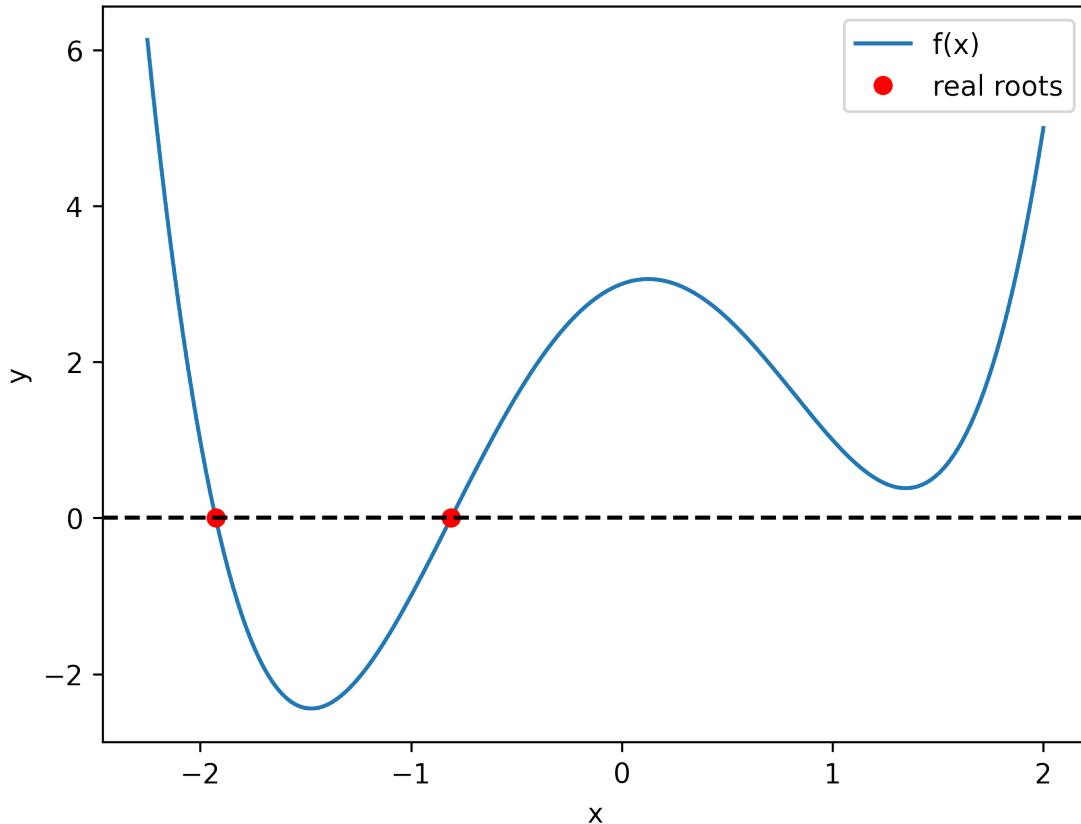
```
import numpy as np
import matplotlib.pyplot as plt

x = np.linspace(-2.25, 2, 200)
y = x**4 - 4*x**2 + x + 3
np.roots([1, 0, -4, 1, 3])
```

```
array([ 1.36778435+0.23154129j,  1.36778435-0.23154129j,
       -1.92630322+0.j          , -0.80926547+0.j          ])
```

There are two real roots to this equation at $x \approx -1.93, -0.81$. We can see these roots here:

```
plt.plot(x, y, label='f(x)')
plt.plot([-1.92630322, -0.80926547], [0, 0], 'ro', label='real roots')
plt.axhline(0, color='k', ls='--')
plt.xlabel('x')
plt.legend()
plt.ylabel('y');
```



To solve this with a generative approach, we first build a generative model. It is tricky to evaluate the goodness of fit in a GMM model. Here we plot predictions with the data for a visual comparison. One can use parity plots, R^2 , MAE, RMSE, and others. We show some of these here. Close inspection of the red predicted line shows some fitting artifacts in the minima and maxima. These do not impact the current problem as they are not near the roots we seek. One could even eliminate that data points where y is not close to zero here.

```
data = np.array([x, y]).T

gmm = best_gmm_bic(data)
pred = gmm.predict([0], data[:, 0:1])

known_y = data[:, 1]
pred_y = pred[:, 0]

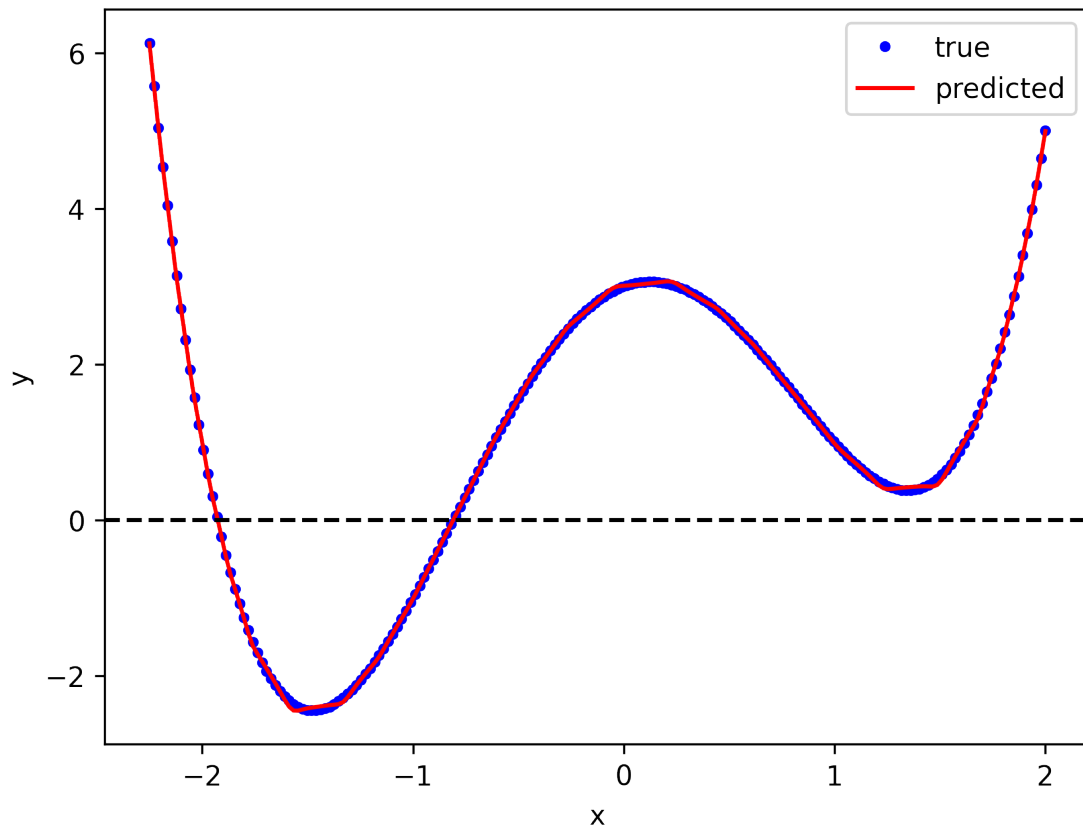
R2 = 1 - np.sum((known_y - pred_y)**2) / np.sum((known_y - np.mean(known_y))**2)

print(f'R2 = {R2:1.3f}')
print(f'MAE = {np.mean(np.abs(known_y - pred_y)):1.2f}')

plt.plot(data[:, 0], data[:, 1], 'b.', label='true')
plt.plot(data[:, 0], pred[:, 0], 'r-', label='predicted')
plt.xlabel('x')
plt.ylabel('y')
plt.axhline(0, c='k', ls='--')
plt.legend();
```

$R2 = 1.000$

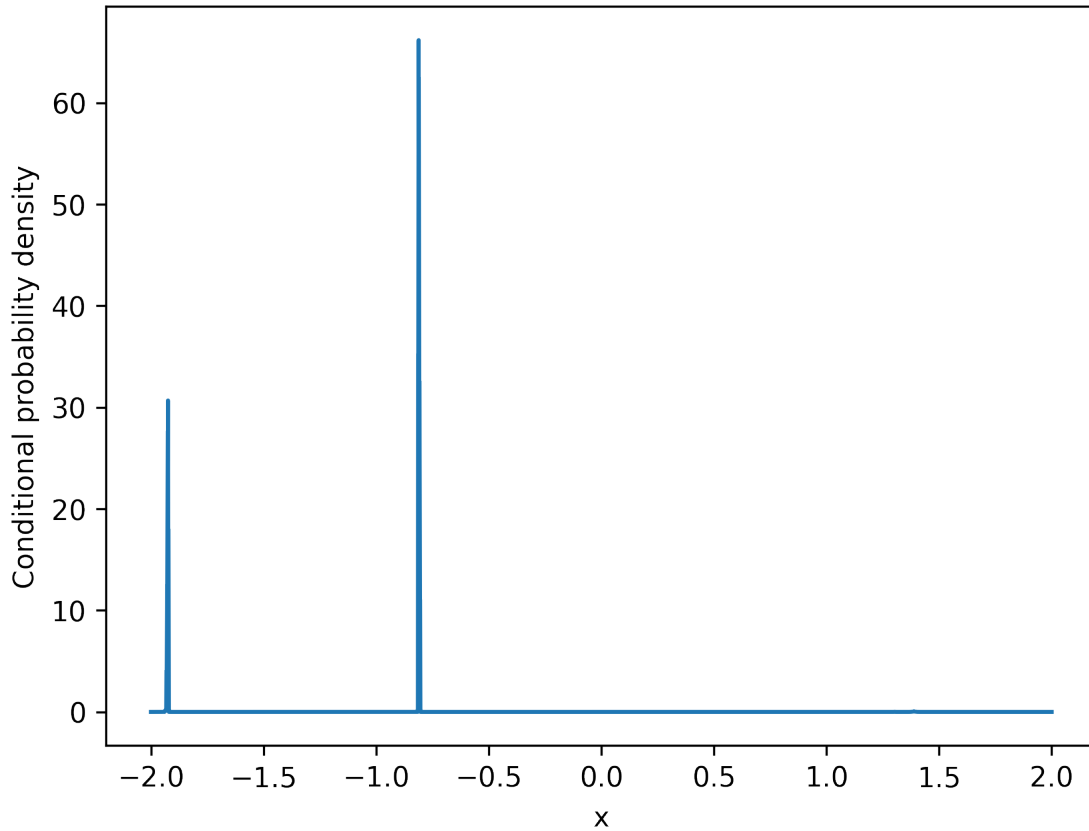
MAE = 0.02



That model fit seems reasonable. Now, we just generate conditional samples for $y = 0$. We can compute the conditional probability density for that condition. Here we can see the probability density is basically zero except for two places that are near the two real roots.

```
# the first argument is the indices to condition on
# the second argument is the value to condition those variables.
# Here x is in index 0, and y is in index 1; [x, y]
c = gmm.condition([1], [[0.0]])
# compute conditional probability that each x corresponds to a value of y=0.0
_x = np.linspace(-2, 2, 1000)[: , None]

plt.plot(_x, c.to_probability_density(_x))
plt.xlabel('x')
plt.ylabel('Conditional probability density');
```



To see a more quantitative result, we have to sample the model many times, and evaluate statistics. Note that this model ends up producing two primary values that correspond to the roots. There is some uncertainty in these values because the model does not fit the data perfectly. That is a limitation of the GMM; there is no noise in our function evaluations.

```
cluster_stats(c.sample(1000))
```

```
0: -1.926 +- 0.0018 (n=310)
1: -0.809 +- 0.0012 (n=689)
-1: 1.377 +- 0.0000 (n=1)
```

Similar to using `scipy.optimize.root`, we cannot find the imaginary roots by sampling real values, and we show how to resolve this in the next section. We note that if the GMM is not sufficiently accurate, it is possible to find other roots; we are finding roots of a surrogate model that approximates the true joint probability distribution. There are sometimes 1-2 samples out of 1000 that is not a solution here, it is labeled -1.

It is also possible to condition on some other value, e.g. what are the x values where $y = 5$. In a conventional root finder one has to define a function equal to zero at that value, but here we simply condition on that value. If we generate enough samples, we get both values. In a conventional root finder, the root obtained depends on the initial guess, and there are basins of attraction around them. There is likely a similar concept here, but accessing this is hidden in the `gmr` library.

```
c = gmm.condition([1], [[5.0]]) # condition on column 1, y=5

results = c.sample(1000)
cluster_stats(results)
```

```
0: -2.206 +- 0.0012 (n=425)
1:  2.000 +- 0.0011 (n=575)
```

Finally, to reinforce that the GMM is a joint distribution model that can "go either way", here we use the GMM as a surrogate for the original function by generating samples conditioned on x . We use the values above to show that we recover the correct x values.

```
# condition on x=2, and x=-2.206
c = gmm.predict([0], [[2.0], [-2.206]])
c
```

```
array([[4.99519103],
       [5.00445111]])
```

For this specific example there is a near analytical way to get the roots using `np.roots` (this function relies on eigenvalues of a companion matrix), and it would be trivial to find the roots with a nonlinear solver very efficiently. There are some advantages to this generative approach over a conventional nonlinear program solver. First, we do not need to make guesses and iterate. Instead, we simply sample until we think we have found enough solutions. This approach does not rely on derivatives in any form. It is also potentially useful that we get a distribution of samples. In this case, that comes from errors in the model, and not from intrinsic noise in the data, but it might help with uncertainty quantification, or in an active learning scenario. Finally, it is also interesting that the GMM can "go both ways", i.e. it is both a forward and backward model relating the inputs and outputs.

An obvious disadvantage, however, is we relied on a lot of function samples, far more than would be required with a nonlinear root finder in this case. It is unclear how little data might be required, or how to develop the most effective set of data points to build this model.

3.1.1 Finding imaginary roots

In the previous example we did not find imaginary roots because we only sampled a real space. We aim here to find imaginary roots. For a concrete example we consider $x^3 - x^2 + x - 1 = 0$. There are three roots: $x = 1, \pm i$. We can express these in complex form as $x = 1 + 0j, 0 + 1j, 0 - 1j$.

```
import numpy as np
r = np.roots([1, -1, 1, -1])
r
```

```
array([ 1.00000000e+00+0.j, -4.23416596e-16+1.j, -4.23416596e-16-1.j])
```

In an array form, those roots look like this. There is a column for the real part, and a column for the imaginary part.

```
with np.printoptions(suppress=True):
    print(np.array([r.real, r.imag]).T)
```

```
[[ 1.  0.]
 [-0.  1.]
 [-0. -1.]]
```

This is the key to formulating this problem. We sample a complex number input space here. This is done simply by sampling the real and imaginary parts separately. We construct the complex array from the samples, and use that to evaluate the function. Finally, we separate the complex results into a real and imaginary columns for the outputs. Sampling is cheap here, so we sample densely in a region we expect to find solutions.

```
inputs = np.random.uniform(-2, 2, (200000, 2))
I = inputs[:, 0] + 1j * inputs[:, 1]
O = I**3 - I**2 + I - 1
outputs = np.array([O.real, O.imag]).T
```

It turns out we do not need all the samples, just the ones that are near the desired values. The desired value here are $[0, 0]$, and we can filter out samples that are far from that. This eliminates many of the samples. There is a balance of choosing a radius that is big enough to get a good set of samples, and not too big to make it difficult to learn the distribution.

```
d = 0.2
ind = np.sum(outputs**2, axis=1) < d**2
print(np.sum(ind))

data = np.hstack([inputs, outputs])[ind]
```

795

Now, we fit and condition on the desired output.

```
gmm = best_gmm_bic(data)

c = gmm.condition([2, 3], [[0.0, 0.0]])

results = c.sample(1000)
plt.plot(*results.T, 'b.', ms=2, alpha=0.2)
plt.xlabel('real')
plt.ylabel('imag');
```

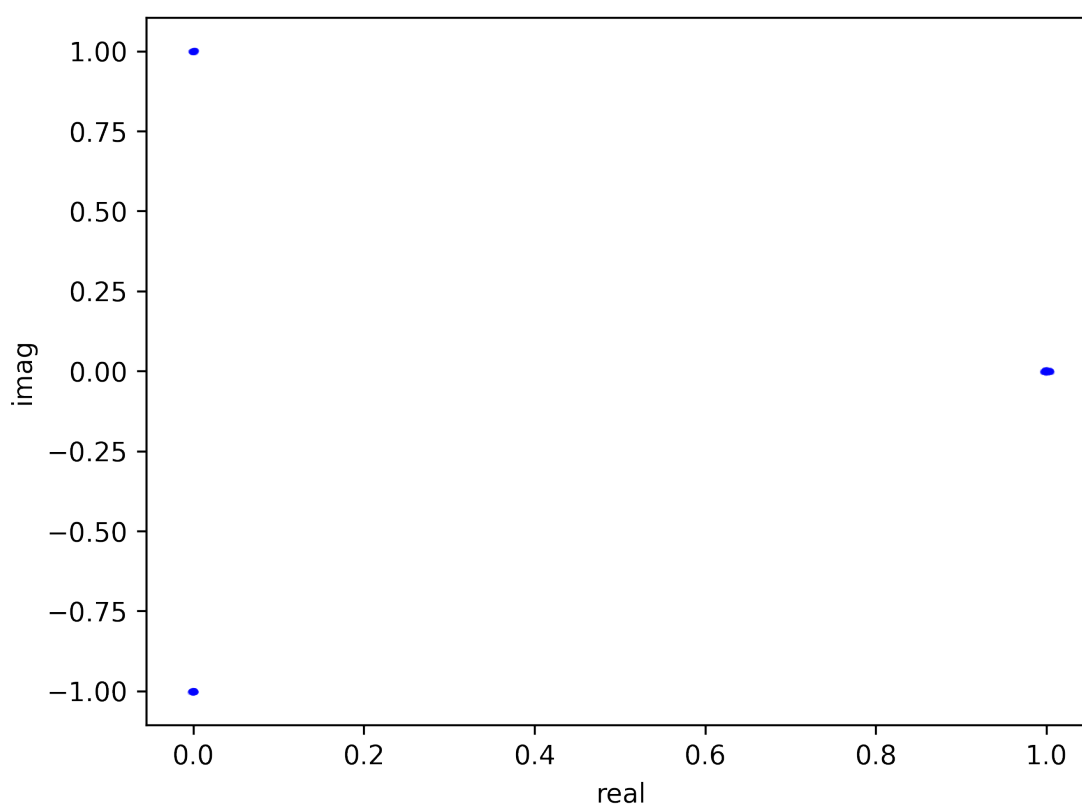
```
/Users/jkitchin/.venv/lib/python3.12/site-packages/sklearn/utils/extmath.py:203: RuntimeWarning:
  ret = a @ b
/Users/jkitchin/.venv/lib/python3.12/site-packages/sklearn/utils/extmath.py:203: RuntimeWarning:
  ret = a @ b
/Users/jkitchin/.venv/lib/python3.12/site-packages/sklearn/utils/extmath.py:203: RuntimeWarning:
  ret = a @ b
```



```

/Users/jkitchin/.venv/lib/python3.12/site-packages/sklearn/utils/extmath.py:203: RuntimeWarning
    ret = a @ b
/Users/jkitchin/.venv/lib/python3.12/site-packages/sklearn/utils/extmath.py:203: RuntimeWarning
    ret = a @ b
/Users/jkitchin/.venv/lib/python3.12/site-packages/sklearn/utils/extmath.py:203: RuntimeWarning
    ret = a @ b
/Users/jkitchin/.venv/lib/python3.12/site-packages/sklearn/utils/extmath.py:203: RuntimeWarning
    ret = a @ b
/Users/jkitchin/.venv/lib/python3.12/site-packages/sklearn/utils/extmath.py:203: RuntimeWarning
    ret = a @ b
/Users/jkitchin/.venv/lib/python3.12/site-packages/sklearn/utils/extmath.py:203: RuntimeWarning
    ret = a @ b

```



We can see there are only three pairs of samples, one at $(1, 0)$, which is the real root, and two at $(0, 1)$ and $(0, -1)$ which represent the pair of imaginary results.

3.2 Optimization with generative models

In the previous examples, we found roots by generating conditional samples where the function had a desired value. It is certainly possible to generate samples and then find the minimum and maximum values in the samples. There is a better way though if we recognize that we can generate derivative samples and then sample by conditioning on a derivative equal to zero.

We continue with the example from Section 3.1 to find the stationary points in $y = x^4 - 4x^2 + x + 3$. The derivative is analytically known: $y' = 4x^3 - 8x + 1$. We show here there are three roots

of the derivative of this function, and we generate samples of the derivative. For a more complex function, a library like `jax` or `numdifftools` could be used for the derivatives.

```
import numpy as np
import matplotlib.pyplot as plt

y_prime = 4*x**3 - 8*x + 1 # this is the derivative of the polynomial
np.roots([4, 0, -8, 1])
```

```
array([-1.4729976 ,  1.34699741,  0.12600019])
```

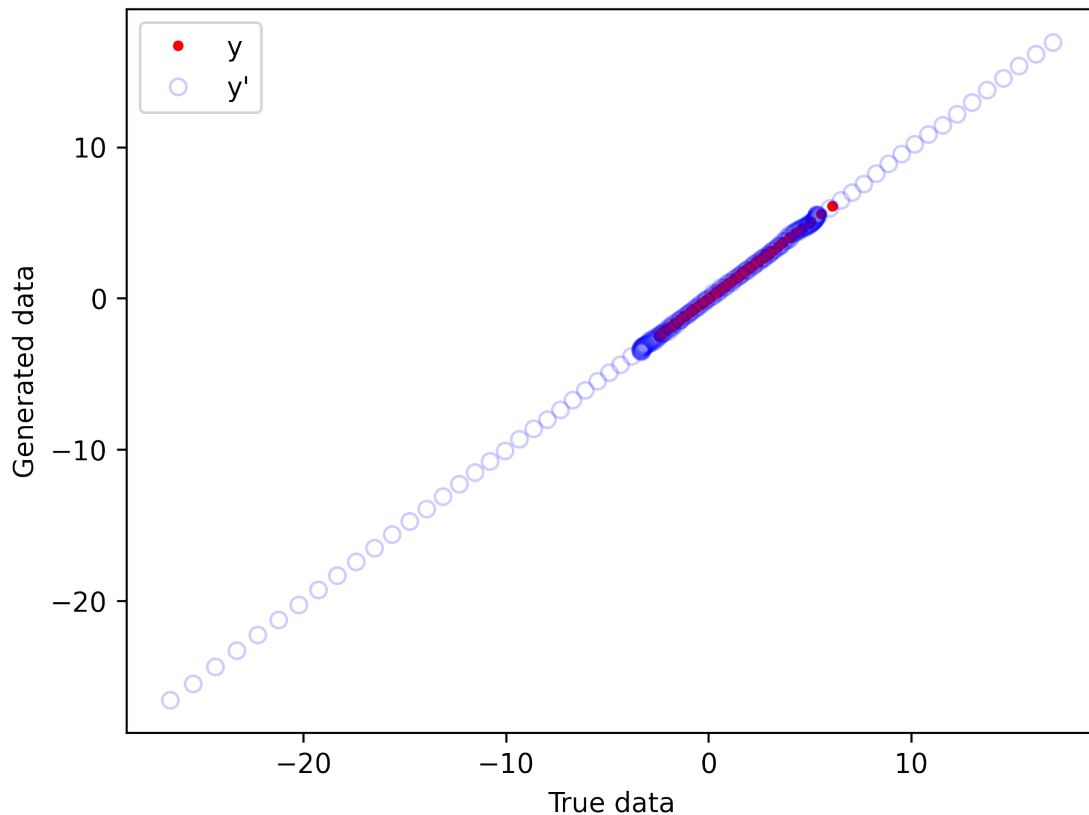
To find these conditional generation, we first build the GMM model for the joint distribution of x , y and y_prime . It is not essential to include y , but one can find (x, y) this way. Here we use a parity plot for a qualitative analysis of the quality of fit.

```
data = np.array([x, y, y_prime]).T

gmm = best_gmm_bic(data)
```

After fitting, we can plot the data to see how well the model works.

```
# condition on x and predict y, y_prime
pred = gmm.predict([0], data[:, 0:1])
plt.plot(data[:, 1], pred[:, 0], 'r.', label='y')
plt.plot(data[:, 2], pred[:, 1], 'bo', mfc='none', alpha=0.2, label="y'")
plt.xlabel('True data')
plt.ylabel('Generated data')
plt.legend();
```



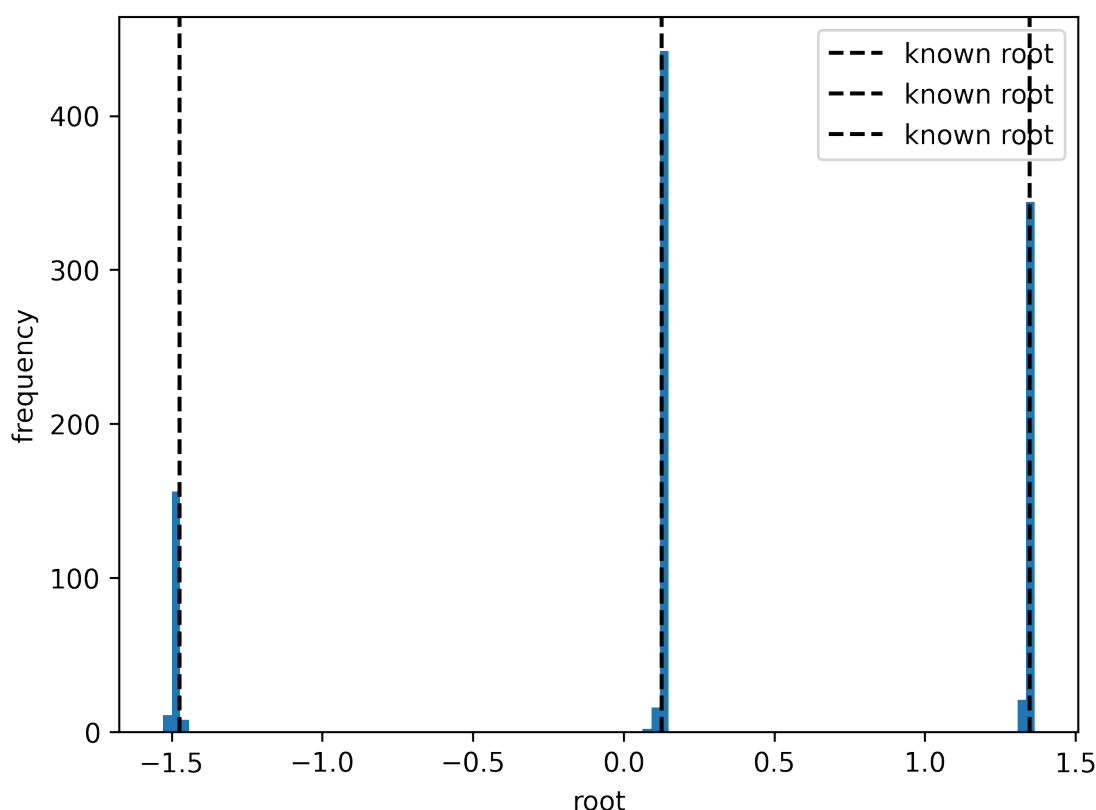
Now we sample for $y' = 0$ and plot the histogram of results. These are pretty close to the analytical solutions. You need a more accurate model (e.g. from more data, and with more Gaussians) to do better.

```
c = gmm.condition([2], [[0.0]])

results = c.sample(1000)
plt.hist(results[:, 0], bins=100)
plt.xlabel('root')
plt.ylabel('frequency')

for r in np.roots([4, 0, -8, 1]):
    plt.axvline(r, c='k', ls='--', label='known root')

plt.legend(loc='best');
```



Note that we find three places where the derivative is zero as expected. However, these correspond to two minima and one maxima. We can further constrain this with samples of the second derivative. We don't know in advance what the value of the Hessian will be, so we cannot directly condition on that. Let us be clever then, and use 1 if second derivative is positive, and -1 if it is negative. Then we condition on $y' = 0$ and $y''_{sign} = -1$. This leads us to one solution, which corresponds to the maximum.

```
y_double_prime = (12 * x**2 - 8) > 0

# Add the integer label for y''
data = np.array([x, y,
                 y_prime,
```

```

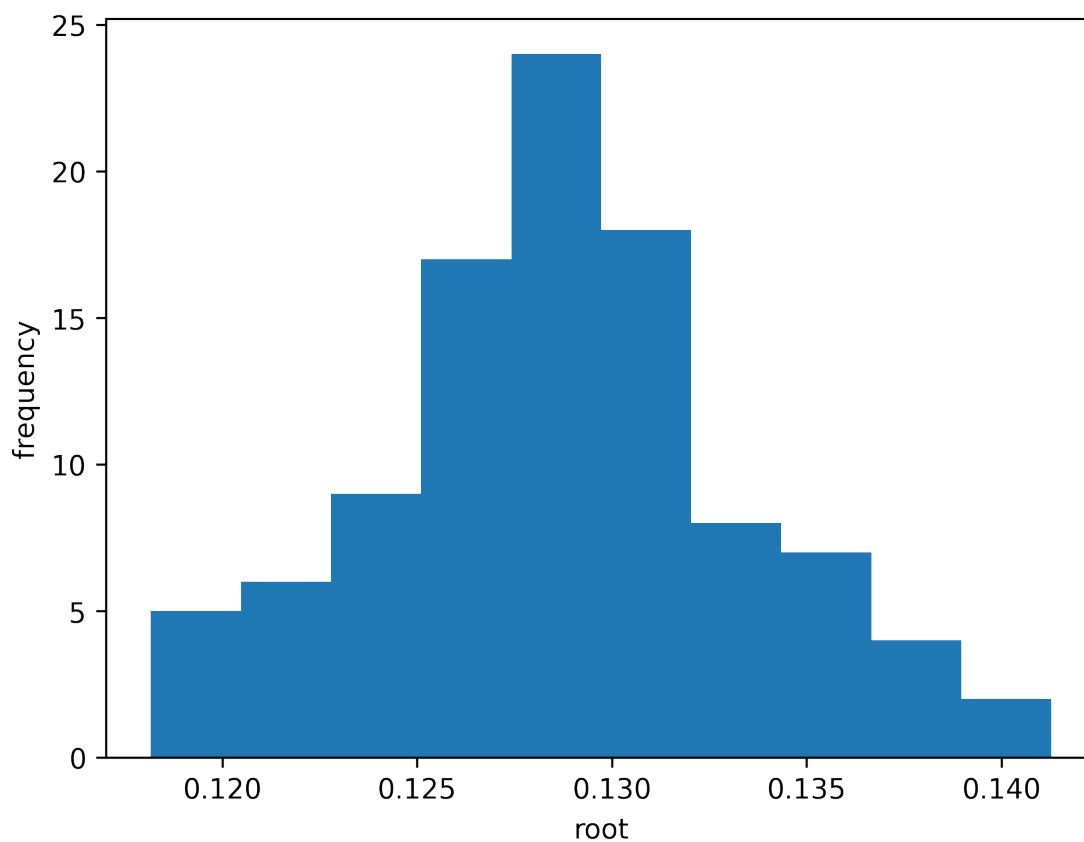
np.where(y_double_prime>0, 1, -1).astype(int)).T

gmm = best_gmm_bic(data)
# condition on y'=0.0, and y''=-1
c = gmm.condition([2, 3], [[0, -1]])

results = c.sample(100)
plt.hist(results[:, 0]) # x
plt.xlabel('root')
plt.ylabel('frequency')
cluster_stats(results[:, 0])

```

-1: 0.129 $\pm$ 0.0046 (n=1)



If we condition instead on $y'' = 1$, we find the minima instead.

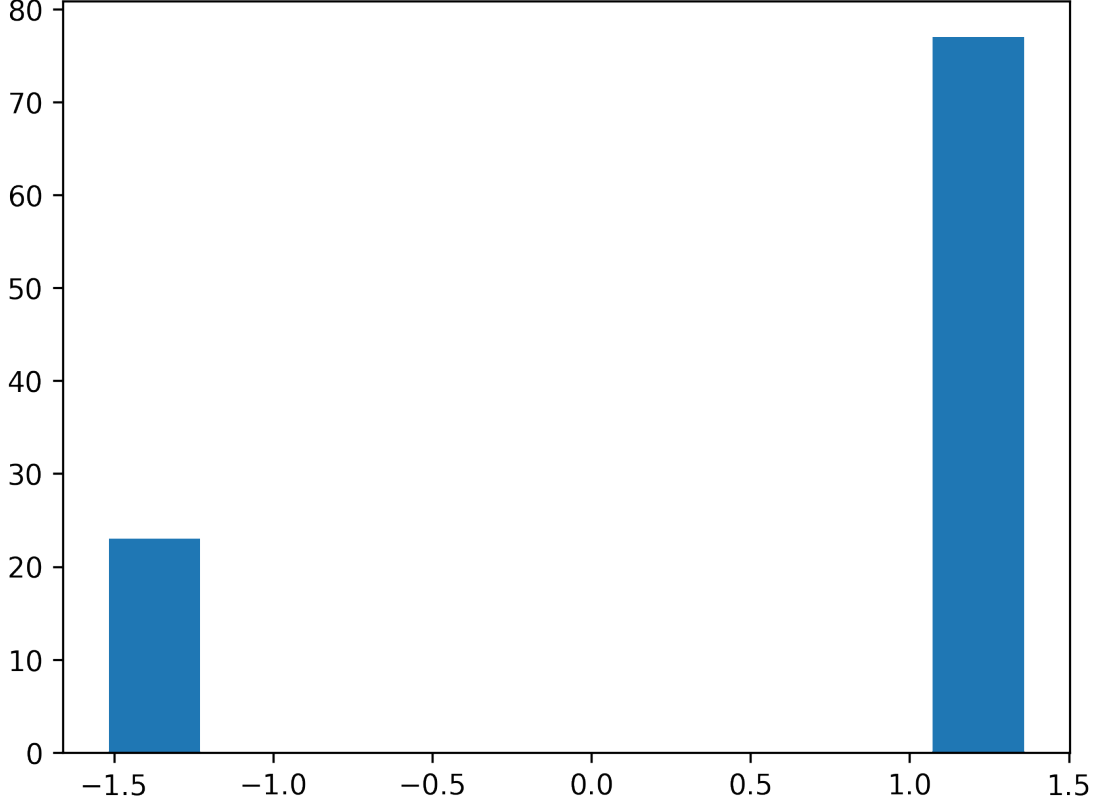
```

c = gmm.condition([2, 3], [[0.0, 1]])

results = c.sample(100)
plt.hist(results[:, 0]);
cluster_stats(results)

```

0: -2.034 $\pm$ 0.5486 (n=23)
1: 0.887 $\pm$ 0.4541 (n=77)



3.3 Optimization with an equality constraint

With an equality constraint it may not be possible to solve this by conditioning on a derivative being zero; if the constraint is active, the derivative(s) of the objective function may not be equal to zero at the solution. To solve this generatively, we turn to a Lagrange function that transforms the constrained problem to an unconstrained problem where the derivatives of the Lagrange function are equal to zero. So we just construct this function, generate samples of the inputs (including the λ values) and the derivatives of the Lagrange function. Then we find solutions by conditioning on $L' = 0$. We will have samples of $x, y, z, \lambda, d\mathcal{L}/dx, d\mathcal{L}/dy, d\mathcal{L}/dz, d\mathcal{L}/d\lambda$.

Formalizing this optimization problem with the generative model predictions as in Eq. 10 in terms of the expected values of the predictions of this model:

$$\begin{aligned} \min_{x,y,z} \quad & \mathbb{E}[f \mid x, y, z; \theta] \\ \text{subject to} \quad & \mathbb{E}[g \mid x, y, z; \theta] = 0 \end{aligned} \tag{10}$$

The GMM is trained on this joint space x_{joint} using the following simplified notation:

$$p(\mathbf{x}_{\text{joint}}) = \sum_{k=1}^K \pi_k \mathcal{N}(\mathbf{x}_{\text{aug}} \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k) \tag{11a}$$

Following the same conditional prediction scheme as in Eq. 9, we compute the expected values of all quantities of interest (such as the objective function f , the equality constraint g), and respective

derivatives conditioned on the variables of choice. In this case, we are interested in the Karush-Kuhn-Tucker (KKT) conditions of the constrained case to be met. To achieve this, we need to define the Lagrangian using GMM-predicted conditional expectations:

$$\mathcal{L}(x, y, z, \lambda; \theta) = \mathbb{E}[f \mid x, y, z; \theta] + \lambda \cdot \mathbb{E}[g \mid x, y, z; \theta] \quad (12)$$

The KKT conditions for optimality are expressed by the Lagrangian gradients with respect to the optimization variables. Each partial derivative is also modeled via conditional inference from the joint GMM:

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial x} &= \mathbb{E}[\partial_x f \mid x, y, z; \theta] + \lambda \cdot \mathbb{E}[\partial_x g \mid x, y, z; \theta] \\ \frac{\partial \mathcal{L}}{\partial y} &= \mathbb{E}[\partial_y f \mid x, y, z; \theta] + \lambda \cdot \mathbb{E}[\partial_y g \mid x, y, z; \theta] \\ \frac{\partial \mathcal{L}}{\partial z} &= \mathbb{E}[\partial_z f \mid x, y, z; \theta] + \lambda \cdot \mathbb{E}[\partial_z g \mid x, y, z; \theta] \\ \frac{\partial \mathcal{L}}{\partial \lambda} &= \mathbb{E}[g \mid x, y, z; \theta] \end{aligned} \quad (13)$$

An interesting paradigm arises here: this constrained optimization problem was just cast as a sampling problem, a conditionally inferred task. We simply query the GMM looking for samples of x, y, z, λ that satisfy the optimality conditions. Formally, we condition on:

$$\frac{\partial \mathcal{L}}{\partial x} = 0, \quad \frac{\partial \mathcal{L}}{\partial y} = 0, \quad \frac{\partial \mathcal{L}}{\partial z} = 0, \quad \frac{\partial \mathcal{L}}{\partial \lambda} = 0 \quad (14)$$

and recover values of the solution that are consistent with the optimality conditions of the original constrained optimization problem.

We illustrate this with this constrained optimization: minimize $x^2 + y^2 + z^2$ subject to the equality constraint defined by the plane $2xy + z = 3$. We first solve this with a conventional solver `scipy.optimize.minimize`.

```
from scipy.optimize import minimize

def objective(X):
    x, y, z = X
    return x**2 + y**2 + z**2

def eq(X):
    x, y, z = X
    return 2 * x - y + z - 3

sol = minimize(objective, [1, -0.5, 0.5], constraints={'type': 'eq', 'fun': eq})
sol
```

```
message: Optimization terminated successfully
success: True
status: 0
  fun: 1.5
   x: [ 1.000e+00 -5.000e-01  5.000e-01]
  nit: 1
  jac: [ 2.000e+00 -1.000e+00  1.000e+00]
 nfev: 4
njev: 1
```

Next we formulate the Lagrange function, and sample x, y, z, λ to generate the data needed for a generative model.

```
# The original objective
def f(X):
    x, y, z = X
    return x**2 + y**2 + z**2

# A constraint function
def g(X):
    x, y, z = X
    return 2 * x - y + z - 3

# The Lagrange function
def L(Y):
    X, _lambda = Y[0:3], Y[3]
    return f(X) + _lambda * g(X)

# Now we generate samples of x,y,z,lambda. Sobol sampling just fills space more
# efficiently than uniform random samples.
from scipy.stats.qmc import Sobol, scale
sobol = Sobol(d=4, scramble=True)
uY = sobol.random(n=128) # unscaled

lower_bounds = [-2, -2, -2, 0]
upper_bounds = [ 2,  1,  1, 1]

Y = scale(uY, l_bounds=lower_bounds, u_bounds=upper_bounds)

# vectorized, element-wise jacobian
from jax import jacobian, vmap
samples = vmap(jacobian(L))(Y)

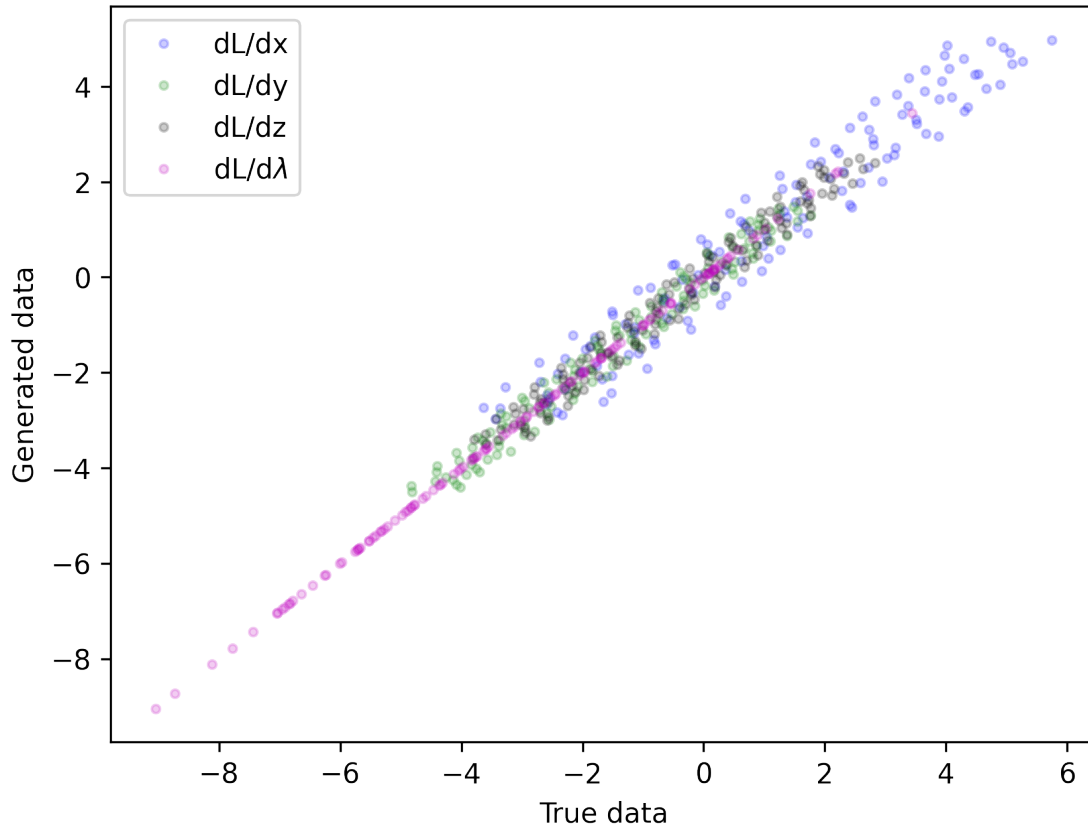
# Here is the data of [x, y, z, lambda, dL/dx, dL/dy, dL/dz, dL/dlambda]
data = np.hstack([Y, samples])
```

Next, we build the GMM.

```
gmm = best_gmm_bic(data)

pred = gmm.predict([0, 1, 2], data[:, 0: 3]) # conditioned on x,y,z

plt.plot(data[:, 4], pred[:, 1], 'b.', label='dL/dx', alpha=0.2)
plt.plot(data[:, 5], pred[:, 2], 'g.', label='dL/dy', alpha=0.2)
plt.plot(data[:, 6], pred[:, 3], 'k.', label='dL/dz', alpha=0.2)
plt.plot(data[:, 7], pred[:, 4], 'm.', label=r'dL/d$\lambda$', alpha=0.2)
plt.xlabel('True data')
plt.ylabel('Generated data')
plt.legend();
```



This model is not great, with scatter about the parity line. It turns out that only a single Gaussian is used. Nevertheless, we can still condition on the Lagrange derivatives being zero to find the optimal solution. This yields the known answer to high accuracy.

```
# condition on dL/dx, dL/dy, dL/dz, dL/dlambda = 0
P = gmm.predict([4, 5, 6, 7], np.array([[0.0, 0.0, 0.0, 0.0]]))
P[0][0:3] # x, y, z
```

```
array([ 0.99999785, -0.49999902,  0.49999868])
```

3.4 Constrained optimization with an inequality

Inequality constraints bring a new challenge to using a generative method; there could be a range of possible values that could satisfy the solution (that is the nature of inequality), so it is not possible to condition on one set of values like it is with the equality constraint. A benefit of the conventional `minimize` function in `scipy` is one simply changes the constraint type, as shown in this example. Here we seek to minimize $(x-2)^2 + (y-1)^2$ subject to $x+y \geq 1$. We first use a conventional solver from `scipy` that encodes the constraint in a function, and declares it an inequality in a dictionary argument to the solver.

```
from scipy.optimize import minimize

def f(X):
```

```

x, y = X
return (x - 2)**2 + (y - 1)**2

def c1(X):
    x, y = X
    return x + y - 1

minimize(f, [1, 1], constraints={'type': 'ineq', 'fun': c1})

```

```

message: Optimization terminated successfully
success: True
status: 0
  fun: 5.551115123125783e-17
   x: [ 2.000e+00  1.000e+00]
  nit: 2
  jac: [ 1.490e-08  0.000e+00]
 nfev: 7
 njev: 2

```

We cannot use the Lagrange method for an inequality constraint the same way we did for an equality constraint. Instead we consider the barrier method.³⁷ Recalling the barrier function for the inequality-constrained optimization problem defined as in Eq. 15, in which the objective function incorporates the set of inequality constraints $\mathcal{I}$ with the form illustrated below.

$$f(x) - \mu \sum_{i \in \mathcal{I}} \log c_i(x) \quad (15)$$

We know from classic optimization that when $\mu \rightarrow 0$ the barrier terms vanishes and the adapted problem in Eq. 15 collapses into the original inequality problem. Now, the main task is to specify a μ value here and that is usually selected iteratively. We do not do that here, but use a value known to be adequate for our proposed approach. It should be possible to iteratively try different values of μ to convergence, but we rather focus on demonstrating that is possible to use a generative approach to inequality constrained optimization problem, as a rich body of literature in inequality constrained optimization already discusses challenges in this area.^{5,37}

The path from here is to create the barrier objective function. Then, we sample a space where we expect to get the answer and evaluate the Jacobian of that function. There are some subtleties in this implementation. We leverage `jax` for the Jacobian. That means we have to use the `jax.numpy` functions in the objective function so that they are differentiable. We still use "regular" numpy functions for the random number generation. Finally we use `vmap` to get an "elementwise" set of derivatives for each input value.

```

import numpy as np
import jax.numpy as jnp
from jax import jacobian, vmap

def barrier_objective(X, mu=1e-7):
    x, y = X
    barrier = -mu * (jnp.log(x + y - 1))
    return f(X) + barrier

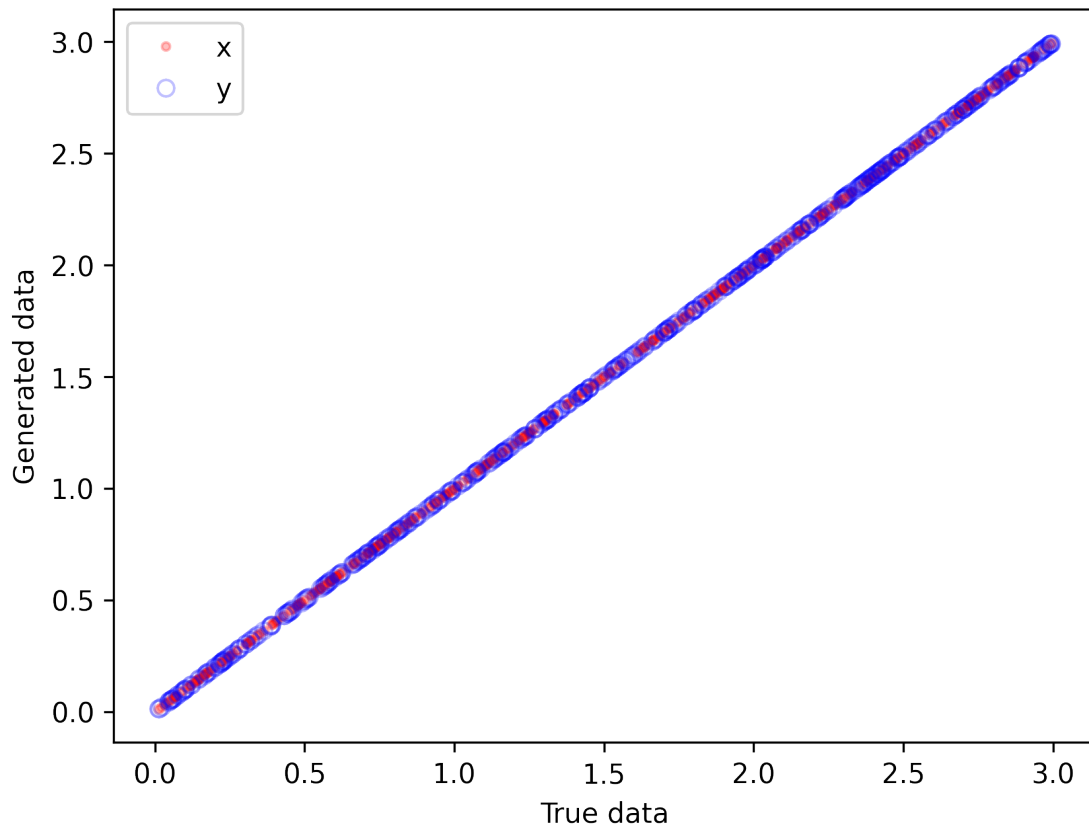
inputs = np.random.uniform(0, 3, (500, 2))
outputs = vmap(jacobian(barrier_objective))(inputs)

data = np.hstack([inputs, outputs])

```

Now, we train the model, and evaluate it. Parity looks fine, so we can proceed to solve the problem.

```
gmm = best_gmm_bic(data)
pred = gmm.predict([2, 3], data[:, 2:])
plt.plot(data[:, 0], pred[:, 0], 'r.', label='x', alpha=0.25)
plt.plot(data[:, 1], pred[:, 1], 'bo', mfc='none', label='y', alpha=0.25)
plt.xlabel('True data')
plt.ylabel('Generated data')
plt.legend();
```



We condition on the outputs (the derivatives of the objective function) being zero and generate inputs.

```
P = gmm.predict([2, 3], np.array([[0.0, 0.0]]))
P
```

```
array([[2.0000003 , 1.00000021]])
```

That is the answer we seek. We again emphasize we make no claims of superiority of the generative solution to the one from `scipy.optimize.minimize`. Indeed, we had to reformulate the problem here, and use a sampling approach. Our primary interest in this problem was simply demonstrating the possibility.

3.5 Parameter estimation with generative models

Parameter estimation in regression is a kind of inverse problem. Usually we consider this in the context of optimization as finding a set of parameters (inputs) that minimize an objective function (outputs) of the errors between a model and observations. To recast this idea for a generative model, we consider the dataset as having columns of parameters (inputs) and observations (outputs). The observations could be derivatives of the objective function, and then one conditions on those being zero like we did before. We take an alternative approach here, and consider the observations to be some output of a model. Then, given an observation, we condition on that to obtain the parameters most consistent with that observation. This approach is fundamentally different than regression where parameters are estimated by minimizing some function of pooled errors.

For a concrete example, we use a series reaction: $A \rightarrow B \rightarrow C$ in a plug flow reactor. The observations we have will be the exit molar flow rates of A and B. We use a inlet molar flow, fixed volume and volumetric flow rate. The system is governed by these two differential equations:

$$\begin{aligned}dF_a/dV &= -k_1 F_a/\nu \\dF_b/dV &= k_1 F_a/\nu - k_2 F_b/\nu\end{aligned}$$

We sample a range of rate constants and compute the exit molar flows of A and B. This data set is what we will use to build a generative model next. We generate samples of k_1, k_2 in the range of 0.1..3.

```
import numpy as np
from scipy.integrate import solve_ivp

nu = 1
Vf = 2
F0 = [1, 0]

def ode(V, F, k1, k2):
    Fa, Fb = F
    dFadV = -k1 * Fa / nu
    dFbdV = k1 * Fa / nu - k2 * Fb / nu
    return dFadV, dFbdV

# Sobol sampling just fills space more efficiently than uniform random samples.
from scipy.stats.qmc import Sobol, scale

sobol = Sobol(d=2, scramble=True)
uK = sobol.random(n=128) # unscaled

lower_bounds = [0.1, 0.1]
upper_bounds = [3, 3]

# scaled, randomly sampled rate constants
K = scale(uK, l_bounds=lower_bounds, u_bounds=upper_bounds)

data = []
for k1, k2 in K:
    sol = solve_ivp(ode, t_span=(0, 2), y0=F0,
                    args=(k1, k2))
    Fa, Fb = sol.y[:, -1]
    data += [[k1, k2, Fa, Fb]]

data = np.array(data)
```

We can see what this data looks like here.

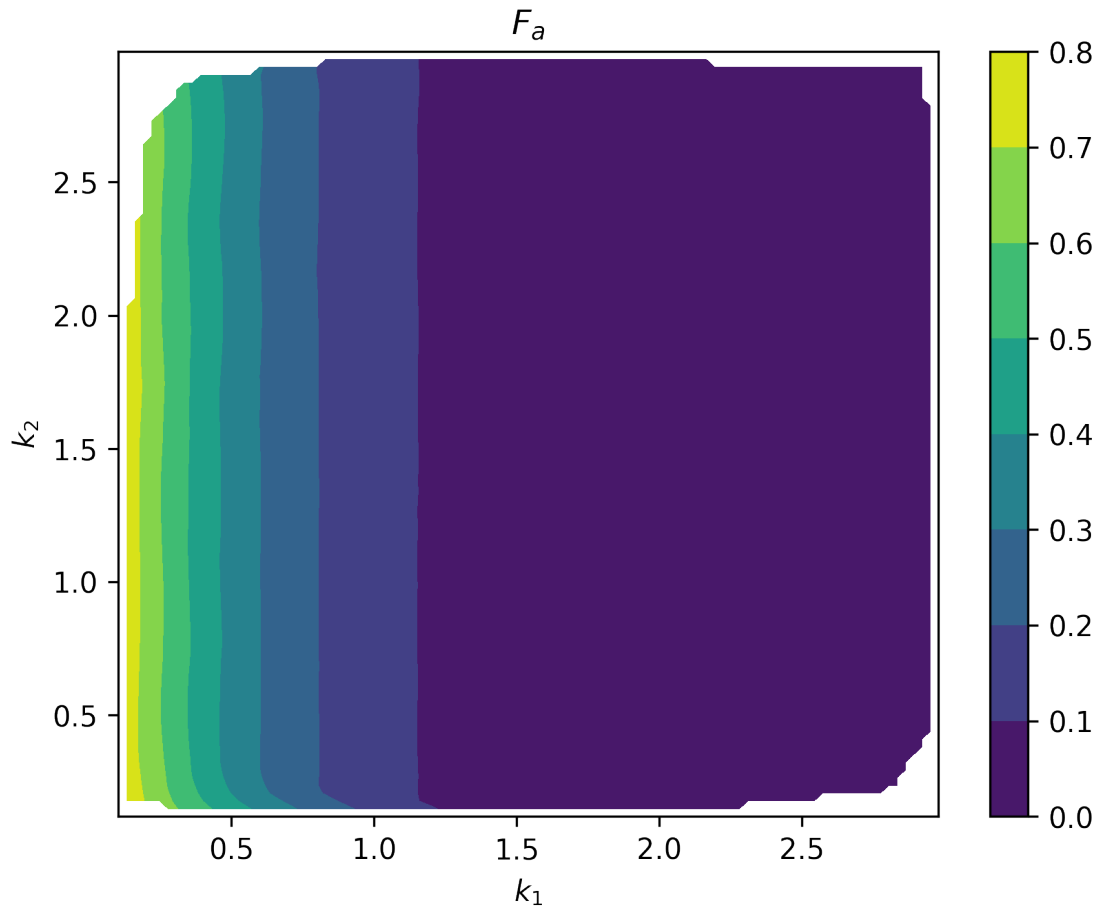
```
from scipy.interpolate import griddata
```

```

xi = np.linspace(data[:, 0].min(), data[:, 0].max(), 100)
yi = np.linspace(data[:, 1].min(), data[:, 1].max(), 100)
X, Y = np.meshgrid(xi, yi)
Z1 = griddata((data[:, 0], data[:, 1]), data[:, 2], (X, Y), method='cubic')
Z2 = griddata((data[:, 0], data[:, 1]), data[:, 3], (X, Y), method='cubic')

plt.contourf(X, Y, Z1)
plt.xlabel('$k_1$')
plt.ylabel('$k_2$')
plt.title('$F_a$')
plt.colorbar();

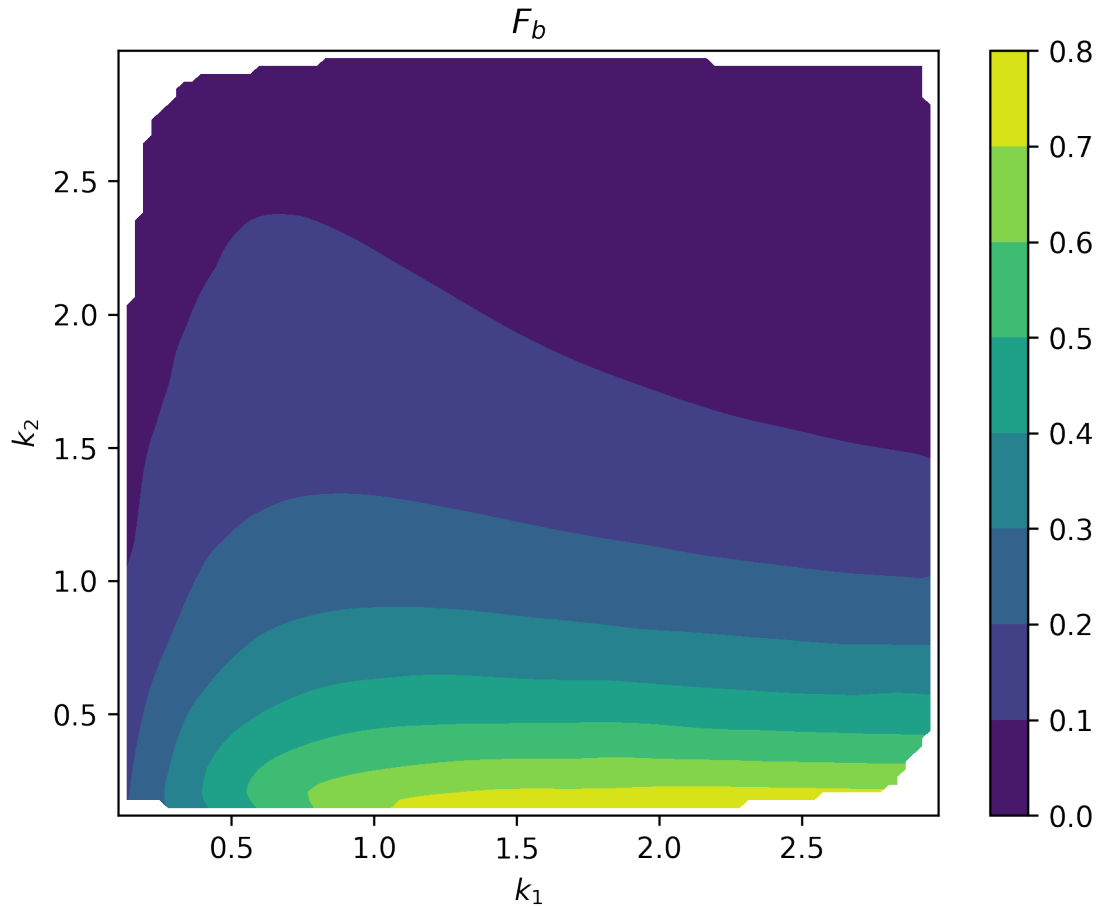
```



```

plt.contourf(X, Y, Z2)
plt.xlabel('$k_1$')
plt.ylabel('$k_2$')
plt.title('$F_b$')
plt.colorbar();

```

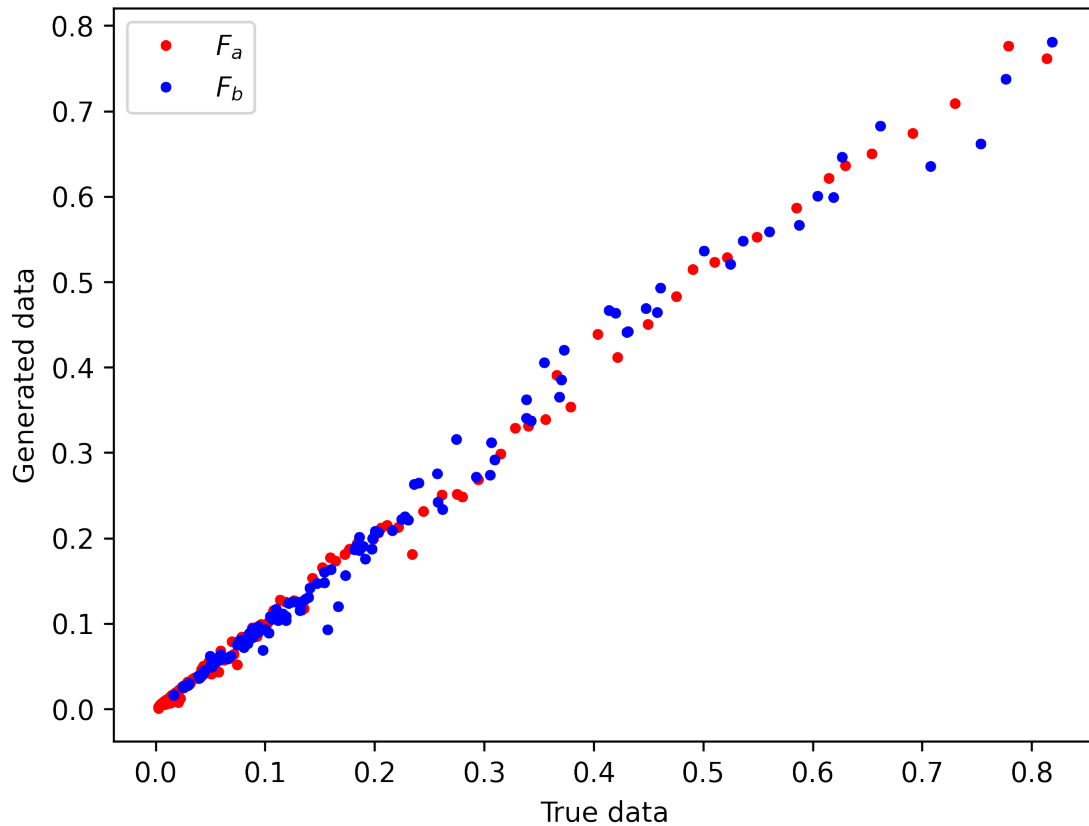


The top figure shows how F_a and F_b vary. It is evidently not enough to specify one of these values to uniquely specify the rate constants. In F_a , there are near vertical isolines, and in F_b other isolines. For example, for $k_1 = 2$, F_a will be between 0-0.02 for a broad range of k_2 values, and which value of k_2 that is chosen depends on the value of F_b that is observed. If you see low F_b , k_2 will be closer to 2, and if F_b is high, then k_2 is closer to 0.2.

Before we can see that, we first fit the generative model.

```
gmm = best_gmm_bic(data)

pred = gmm.predict([0, 1], data[:, 0:2])
plt.plot(data[:, 2], pred[:, 0], 'r.', label='$F_a$')
plt.plot(data[:, 3], pred[:, 1], 'b.', label='$F_b$')
plt.xlabel('True data')
plt.ylabel('Generated data')
plt.legend();
```



Now we estimate parameters for two scenarios at the exit:

1. A low molar flow of A with low molar flow of B, i.e. both reactions are fast.
2. A low molar flow of A with high molar flow of B, i.e. reaction 1 is fast, and reaction 2 is slow.

```
P = gmm.predict([2, 3], np.array([[0.02, 0.05],
                                   [0.02, 0.65]]))
P
```

```
array([[1.96512053, 2.41770378],
       [1.92437692, 0.30988341]])
```

Finally, we can check this result. Here is the case for low molar flows of A and low molar flows of B. This means that k_2 must be high, so that B is consumed. That is what we observe.

```
Fa_exit, Fb_exit = solve_ivp(ode, t_span=(0, 2), y0=F0,
                             args=P[0]).y[:, -1]

with np.printoptions(precision=2):
    print(f'k={P[0]}, Fa={Fa_exit:1.3f}, Fb={Fb_exit:1.3f}')
```

```
k=[1.97 2.42], Fa=0.020, Fb=0.051
```

Here is the case for low concentration of A and high concentration of B. That means k_2 must be low, and it is, and so B accumulates.

```
Fa_exit, Fb_exit = solve_ivp(ode, t_span=(0, 2), y0=F0,
                             args=P[1]).y[:, -1]

with np.printoptions(precision=2):
    print(f'k={P[1]}, Fa={Fa_exit:1.3f}, Fb={Fb_exit:1.3f}')
```

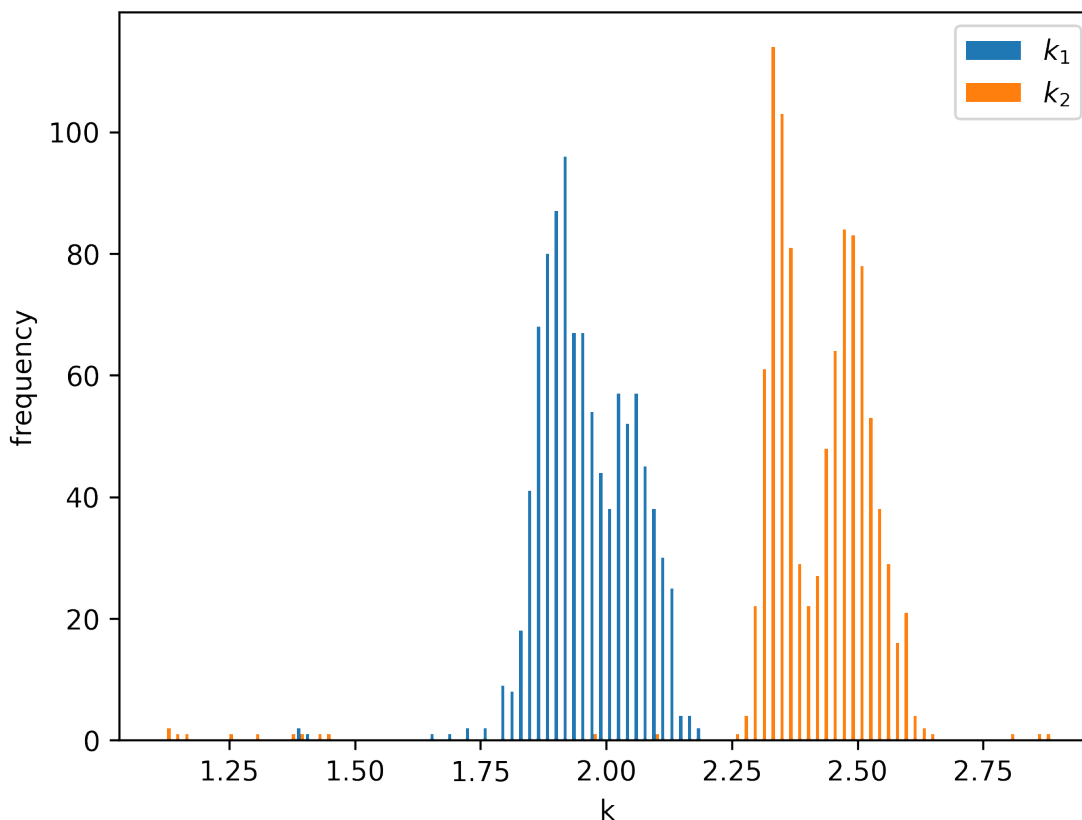
$k=[1.92 \ 0.31]$, $Fa=0.021$, $Fb=0.616$

It is also possible to estimate an uncertainty of sorts. We reconsider the case where there is a low molar flow of A (0.02) with low molar flow of B (0.05) at the exit. We note "of sorts" because in this case there is no noise in the data, and the spread of

```
c = gmm.condition([2, 3], np.array([[0.02, 0.05]]))
ks = c.sample(1000)
plt.hist(ks, bins=100)
plt.legend(['$k_1$', '$k_2$'])
plt.xlabel('k')
plt.ylabel('frequency')

# statistics
m, s = np.mean(ks, axis=0), np.std(ks, axis=0)
print(f'k_1 = {m[0]:1.2f} +/- {s[0]:1.2f}')
print(f'k_2 = {m[1]:1.2f} +/- {s[1]:1.2f}')
```

$k_1 = 1.97 \ +/- \ 0.09$
 $k_2 = 2.42 \ +/- \ 0.15$



We reiterate that the distribution here is due to the GMM surrogate model of the distribution not fitting the data perfectly well, as seen in the parity plot. It is also possible to use this approach to map uncertainties in the inputs to the outputs and vice versa. For example, if one has the confidence intervals of a parameter they can map that range onto outputs with this approach. We illustrate this kind of mapping in Sec. ??.

3.5.1 Caveats

It is not reasonable to think that this approach would work for observations outside the data range. This is akin to extrapolation, and these models are not reliable for that. Here we show two kinds of extrapolation. The first is an observation of negative concentration, which is an unphysical observation. In the second example, we specify a value of B outside the observed data range. Both cases lead to estimated parameters. In the first case, the parameters do not lead to the conditioned outputs. In the second case, an unphysically negative rate constant is predicted.

```
P = gmm.predict([2, 3], np.array([[0.02, -0.5],
                                   [0.02, 2]]))
```

P

```
array([[ 1.85774868,  1.98355625],
       [ 1.98137789, -1.75337919]])
```

```
Fa_exit, Fb_exit = solve_ivp(ode, t_span=(0, 2), y0=F0,
                             args=P[1]).y[:, -1]
```

```
with np.printoptions(precision=2):
    print(f'k={P[1]}, Fa={Fa_exit:1.3f}, Fb={Fb_exit:1.3f}')
```

```
k=[ 1.98 -1.75], Fa=0.019, Fb=17.678
```

There is not even really a clue from the distribution here that something is wrong with that negative rate constant. Here we generate 1000 samples, and see that the distribution is fairly narrow compared to the predicted value. Note in this case, there is no noise in the data, so the distribution of predicted values here comes solely from the approximated distribution.

```
c = gmm.condition([2, 3], [[0.02, 2]])

results = c.sample(1000)
with np.printoptions(precision=4):
    print(f'Mean: {np.mean(results, axis=0)[0:3]}')
    print(f'Std: {np.std(results, axis=0)[0:3]}')
```

```
Mean: [ 1.9821 -1.7525]
Std:  [0.1071 0.0438]
```

3.6 Mapping input and output spaces

We previously developed a mapping approach based on differential equations.³⁸ We illustrate here that this can also be achieved by a generative model. We expand this idea to using a generative model here. The goal is to map a desired output space in the concentration of A, B from a CSTR to the input space of the radius of a CSTR, and the temperature (in units of RT) that achieves it. We have a design with fixed height, and we can modify the radius and temperature (via RT). There are two reactions: $A \rightarrow B$, that is second order in A, and $A \rightarrow C$, that is first order in A. We seek a design space with desired amounts of A and B in the exit. We choose a circular region in the output space, and map it to the input space.

Given a radius, we have $V = \pi R^2 H$ and given an RT we have these rate constants for the two forward reactions: $k_1 = \exp(-3/RT)$, $k_2 = \exp(-10/RT)$.

We start with a mole balance:

$$0 = F_{A0} - F_A + r_A V$$

Some rearrangement leads to:

$$0 = V k_1 C_A^2 + (v_0 + V k_2) C_A - F_{A0}$$

which we can solve with the quadratic equation to get C_A as a function of (R, RT) , and then from a mole balance on B, we get

$$C_B = k_1 C_A^2 V / v_0$$

Thus, we can analytically map the input space to the output space. We do this by brute force sampling here and show how a circular region in the output space maps to the input space. We choose this example because it is easy to do this analytically, but this forward mapping can also be done in nonlinear programs. A downside here is the "low" density of samples does not make the desired region especially well defined.

```

import numpy as np
from scipy.stats.qmc import Sobol, scale

sobol = Sobol(d=2, scramble=True)
uX = sobol.random(n=512) # unscaled

lower_bounds = [0.1, 0.1]
upper_bounds = [3, 15]
X = scale(uX, l_bounds=lower_bounds, u_bounds=upper_bounds)

R, RT = X.T
H = 1
V = np.pi * R**2 * H
k1 = np.exp(-3 / RT)
k2 = np.exp(-10 / RT)

vo = 1
Cao = 1

a = V * k1
b = vo + V * k2
c = -vo * Cao

Ca = (-b + np.sqrt(b**2 - 4 * a * c)) / (2 * a)

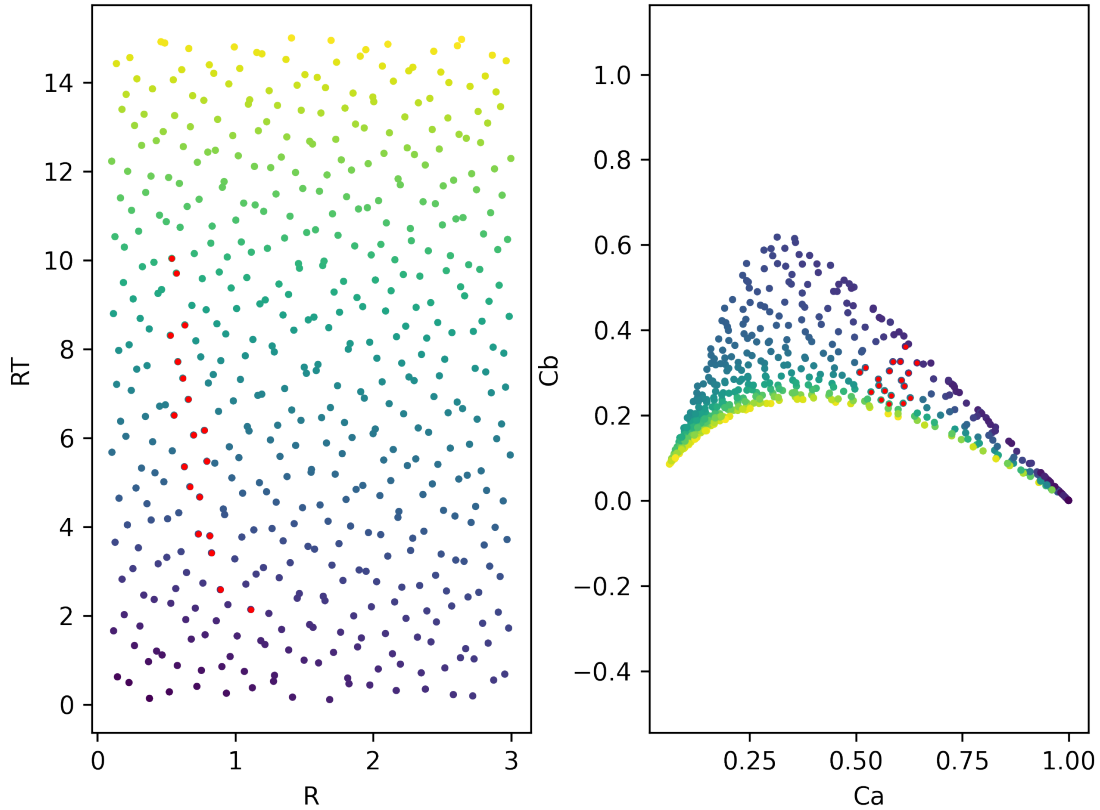
Cb = k1 * Ca**2 * V / vo

import matplotlib.pyplot as plt
plt.subplot(121)
plt.scatter(R, RT, s=4, c=np.sqrt(R**2 + RT**2))
plt.xlabel('R')
plt.ylabel('RT')

plt.subplot(122)
plt.scatter(Ca, Cb, s=4, c=np.sqrt(R**2 + RT**2))
plt.xlabel('Ca')
plt.ylabel('Cb')
plt.tight_layout()

radius = 0.08
dos = ((Ca - 0.58)**2 + (Cb - 0.3)**2) <= radius**2
plt.plot(Ca[dos], Cb[dos], 'r.', ms=2)
plt.axis('equal')
plt.subplot(121)
plt.plot((V[dos] / np.pi / H)**0.5, RT[dos], 'r.', ms=3);

```



To solve this generatively, we build a GMM from the data.

```
data = np.array([R, RT, Ca, Cb]).T
gmm = best_gmm_bic(data)
```

Now we can generate more samples in the desired range. The output circle we desire is centered at (0.58, 0.3) with a radius of 0.08. We generate a lot of samples in this region.

```
r = 0.08
x1 = np.linspace(0.58 - r, 0.58 + r, 500)
x2 = np.linspace(0.3 - r, 0.3 + r, 500)
X1, X2 = np.meshgrid(x1, x2)
X1 = X1.flatten()
X2 = X2.flatten()

pred = gmm.predict([2, 3], np.array([X1, X2]).T)

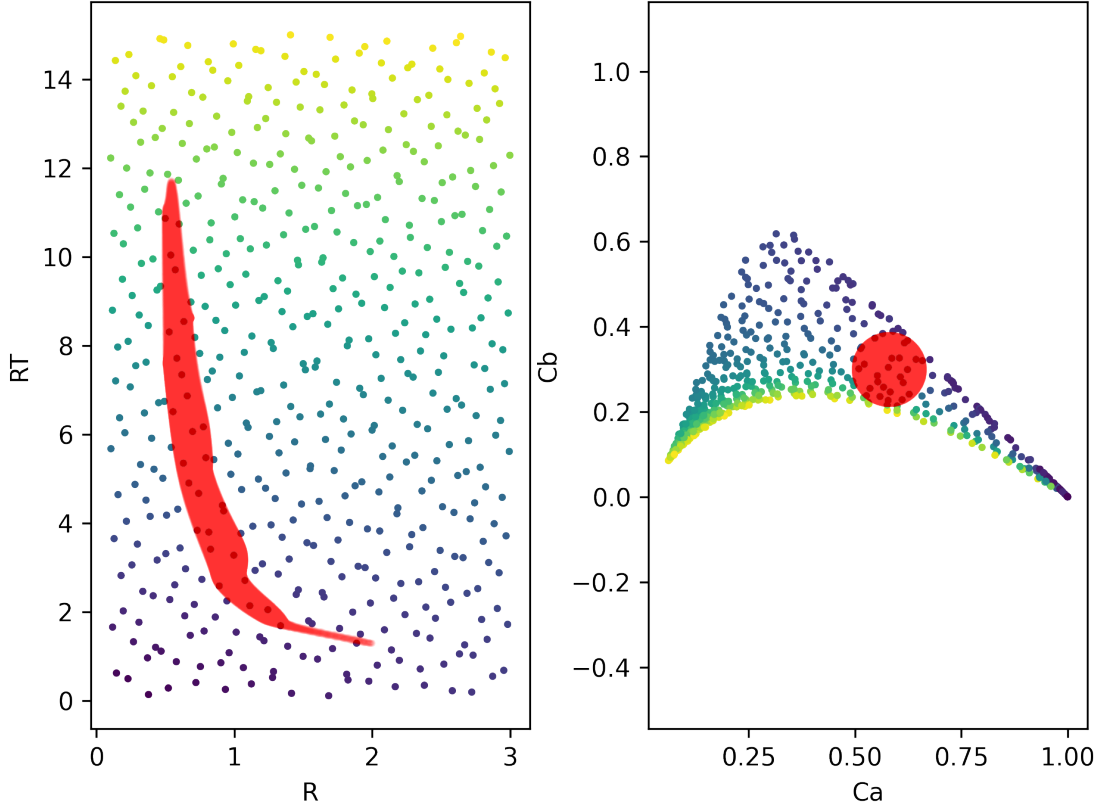
# index variable to select points in a circle centered at (0.58, 0.3) with
# radius 0.08
ind = (X1 - 0.58)**2 + (X2 - 0.3)**2 <= 0.08**2

P1, P2 = pred[ind].T

plt.subplot(121)
plt.scatter(R, RT, s=4, c=np.sqrt(R**2 + RT**2))
plt.xlabel('R')
plt.ylabel('RT')
plt.scatter(P1, P2, s=4, c='r', alpha=0.02)

plt.subplot(122)
```

```
plt.scatter(Ca, Cb, s=4, c=np.sqrt(R**2 + RT**2) )
plt.xlabel('Ca')
plt.ylabel('Cb')
plt.scatter(X1[ind], X2[ind], c='r', s=4, alpha=0.02)
plt.tight_layout()
plt.axis('equal');
```



This is not the same as learning a forward or backward model, e.g. a neural network or Gaussian process model. This is an approximate, data-driven mapping through a joint distribution model. It is not as smooth as the mapping by ODEs, or an analytical mapping, but it might be suitable for some applications. Building the model from more samples would improve the mapping accuracy. It is also possible to map in the forward direction, e.g. from a desired input space to an output space. This has application in uncertainty propagation.

We think there is a connection between the mapping of input and output spaces by differential equations we previously developed³⁸ and modern diffusion and continuous flow mapping (CFM) generative models. We used a differentiable model to derive a set of ODEs that connect the inputs and outputs. We considered a point-wise mapping where one could use the ODEs to transform a single point in one space to the other space. In diffusion and CFM models, one uses machine learning to learn the vector field that defines the ODEs that transform the input distribution to the output distribution. Then, by sampling from one space it is possible to transform that sample into the other space. In this way, we might view these models as data-driven versions of the model-driven mapping idea we developed.

4 Conclusions

We have illustrated several different examples of using generative models in optimization tasks that ranged from root finding, optimization, optimization with constraints, and parameter estimation. In each case the solution is obtained by conditional generation from a distribution trained from samples of inputs and outputs. In root-finding, the outputs are function values. In optimization, the outputs include derivatives. For constrained optimization, the outputs were derivatives of an augmented Lagrange function. In parameter estimation, we used observations for the outputs. The identification of the input / output pairs that allow one to generate conditional samples from specific values is the key to using generative models this way.

There are a few ways this generative approach could fail. First, if it is hard to model the distribution, or learn the transformation the approach would obviously fail. Second, even if it succeeds, there is no guarantee one finds all the possible solutions, even of the surrogate function. The surrogate function may not have sufficient accuracy to find the true solutions of the real system, or may have artifacts that lead to false solutions. The models are built from and the solutions are found by sampling. For well-behaved broad distributions, sampling should work very well. For poorly-behaved distributions, e.g. where there is a very sharp probability, it is possible to miss a result, both in building the model and sampling it for solutions.

It is evident this is not a panacea solution to optimization. Conventional methods may be more efficient and accurate for some problems, especially when there is a model available. Even regular machine learned models with conventional methods may be better when they have desirable properties, e.g. converting a Relu neural network to a mixed integer program. It remains unknown if we can find a way to include inequality constraints. The whole approach resembles using a surrogate model, and is limited in the same ways; the solutions are to the surrogate model, and not the underlying true model. That is a feature when there is no actual model, but rather data. Sampling strategies can be essential to success, and there are not yet established methods to efficiently sample the data one needs. Since this is ultimately a data-driven approach, it is not likely to be accurate in sample generation with conditions that are out of distribution.

Generative models currently do not have the same tools for analysis that regular models have. It is not obvious, for example, how one might define the derivatives of a generative model. In principle, the derivative would itself be a distribution. As with other machine learning models, data-driven generative models should only be expected to be valid for "in-domain" sampling, but in high-dimensional spaces it is not always obvious how to quantify what is in or out of domain. Although one can use sampling to quantify variance in predictions, that variance is a combination of model inadequacy and noise in the data.

Nevertheless, this work shows that generative methods have enormous possibility in a broad range of applications we have not previously considered them for in optimization, as surrogate models, and likely others we have not thought of yet. There are many generative models in existence today, and new ones are still being found. It seems likely these will mitigate problems we have illustrated in this work, and make new approaches possible in the future. We are most excited by the new way of thinking about problems that this approach now enables.

5 Acknowledgments

JRK acknowledges Carl Laird for suggesting the barrier function approach for problems with inequality constraints.

References

- [1] George B Dantzig. Origins of the simplex method. In *A history of scientific computing*, pages 141–151. 1990.
- [2] S. P. Han. A globally convergent method for nonlinear programming. *Journal of Optimization Theory and Applications*, 22(3):297–309, Jul 1977. ISSN 1573-2878. doi: 10.1007/BF00932858. URL <https://doi.org/10.1007/BF00932858>.
- [3] M. J. D. Powell. A fast algorithm for nonlinearly constrained optimization calculations. In G. A. Watson, editor, *Numerical Analysis*, pages 144–157, Berlin, Heidelberg, 1978. Springer Berlin Heidelberg. ISBN 978-3-540-35972-2.
- [4] Robert B. Wilson. *A Simplicial Algorithm for Concave Programming*. Doctor of business administration dissertation, Harvard Business School, Cambridge, MA, 1963. URL <https://nrs.harvard.edu/URN-3:HUL.INSTREPOS:37372525>. Discipline: Business Administration. Advisor: Colin Lukens.
- [5] Andreas Wächter and Lorenz T Biegler. On the implementation of an interior-point filter line-search algorithm for large-scale nonlinear programming. *Mathematical programming*, 106: 25–57, 2006.
- [6] Pierre Bonami, Lorenz T. Biegler, Andrew R. Conn, Gérard Cornuéjols, Ignacio E. Grossmann, Carl D. Laird, Jon Lee, Andrea Lodi, François Margot, Nicolas Sawaya, and Andreas Wächter. An algorithmic framework for convex mixed integer nonlinear programs. *Discrete Optimization*, 5(2):186–204, 2008. ISSN 1572-5286. doi: <https://doi.org/10.1016/j.disopt.2006.10.011>. URL <https://www.sciencedirect.com/science/article/pii/S1572528607000448>. In Memory of George B. Dantzig.
- [7] Ignacio E. Grossmann and Juan P. Ruiz. Generalized disjunctive programming: A framework for formulation and alternative algorithms for minlp optimization. In Jon Lee and Sven Leyffer, editors, *Mixed Integer Nonlinear Programming*, pages 93–115, New York, NY, 2012. Springer New York. ISBN 978-1-4614-1927-3.
- [8] C.A. Floudas, I.G. Akrotirianakis, S. Caratzoulas, C.A. Meyer, and J. Kallrath. Global optimization in the 21st century: Advances and challenges. *Computers & Chemical Engineering*, 29(6):1185–1202, 2005. ISSN 0098-1354. doi: <https://doi.org/10.1016/j.compchemeng.2005.02.006>. URL <https://www.sciencedirect.com/science/article/pii/S0098135405000104>. Selected Papers Presented at the 14th European Symposium on Computer Aided Process Engineering.
- [9] C. A. Floudas and C. E. Gounaris. A review of recent advances in global optimization. *Journal of Global Optimization*, 45(1):3–38, Sep 2009. ISSN 1573-2916. doi: 10.1007/s10898-008-9332-8. URL <https://doi.org/10.1007/s10898-008-9332-8>.
- [10] Fani Boukouvala, Ruth Misener, and Christodoulos A. Floudas. Global optimization advances in mixed-integer nonlinear programming, minlp, and constrained derivative-free optimization, cdfo. *European Journal of Operational Research*, 252(3):701–727, 2016. ISSN 0377-2217. doi: <https://doi.org/10.1016/j.ejor.2015.12.018>. URL <https://www.sciencedirect.com/science/article/pii/S037722171501142X>.

- [11] Pauli Virtanen, Ralf Gommers, Travis E. Oliphant, Matt Haberland, Tyler Reddy, David Cournapeau, Evgeni Burovski, Pearu Peterson, Warren Weckesser, Jonathan Bright, Stéfan J. van der Walt, Matthew Brett, Joshua Wilson, K. Jarrod Millman, Nikolay Mayorov, Andrew R. J. Nelson, Eric Jones, Robert Kern, Eric Larson, C J Carey, İlhan Polat, Yu Feng, Eric W. Moore, Jake VanderPlas, Denis Laxalde, Josef Perktold, Robert Cimrman, Ian Henriksen, E. A. Quintero, Charles R. Harris, Anne M. Archibald, Antônio H. Ribeiro, Fabian Pedregosa, Paul van Mulbregt, SciPy 1.0 Contributors, Aditya Vijaykumar, Alessandro Pietro Bardelli, Alex Rothberg, Andreas Hilboll, Andreas Kloeckner, Anthony Scopatz, Antony Lee, Ariel Rokem, C. Nathan Woods, Chad Fulton, Charles Masson, Christian Häggström, Clark Fitzgerald, David A. Nicholson, David R. Hagen, Dmitrii V. Pasechnik, Emanuele Olivetti, Eric Martin, Eric Wieser, Fabrice Silva, Felix Lenders, Florian Wilhelm, G. Young, Gavin A. Price, Gert-Ludwig Ingold, Gregory E. Allen, Gregory R. Lee, Hervé Audren, Irvin Probst, Jörg P. Dietrich, Jacob Silterra, James T Webber, Janko Slavič, Joel Nothman, Johannes Buchner, Johannes Kulick, Johannes L. Schönberger, José Vinícius de Miranda Cardoso, Joscha Reimer, Joseph Harrington, Juan Luis Cano Rodríguez, Juan Nunez-Iglesias, Justin Kuczynski, Kevin Tritz, Martin Thoma, Matthew Newville, Matthias Kümmerer, Maximilian Bolingbroke, Michael Tartre, Mikhail Pak, Nathaniel J. Smith, Nikolai Nowaczyk, Nikolay Shebanov, Oleksandr Pavlyk, Per A. Brodtkorb, Perry Lee, Robert T. McGibbon, Roman Feldbauer, Sam Lewis, Sam Tygier, Scott Sievert, Sebastiano Vigna, Stefan Peterson, Surhud More, Tadeusz Pudlik, Takuya Oshima, Thomas J. Pingel, Thomas P. Robitaille, Thomas Spura, Thouis R. Jones, Tim Cera, Tim Leslie, Tiziano Zito, Tom Krauss, Utkarsh Upadhyay, Yaroslav O. Halchenko, and Yoshiki Vázquez-Baeza. Scipy 1.0: Fundamental algorithms for scientific computing in python. *Nature Methods*, 17(3):261–272, 2020. doi: 10.1038/s41592-019-0686-2. URL <https://doi.org/10.1038/s41592-019-0686-2>.
- [12] Michael L. Bynum, Gabriel A. Hackebeil, William E. Hart, Carl D. Laird, Bethany L. Nicholson, John D. Sirola, Jean-Paul Watson, and David L. Woodruff. *Pyomo - Optimization Modeling in Python*. Springer Optimization and Its Applications. Springer International Publishing, 2021. doi: 10.1007/978-3-030-68928-5. URL <https://doi.org/10.1007/978-3-030-68928-5>.
- [13] Logan Beal, Daniel Hill, R Martin, and John Hedengren. Gekko optimization suite. *Processes*, 6(8):106, 2018. doi: 10.3390/pr6080106.
- [14] Gurobi Optimization, LLC. Gurobi Optimizer Reference Manual, 2024. URL <https://www.gurobi.com>.
- [15] Erling D. Andersen and Knud D. Andersen. *The Mosek Interior Point Optimizer for Linear Programming: An Implementation of the Homogeneous Algorithm*, pages 197–232. Springer US, Boston, MA, 2000. ISBN 978-1-4757-3216-0. doi: 10.1007/978-1-4757-3216-0_8. URL https://doi.org/10.1007/978-1-4757-3216-0_8.
- [16] Iain Dunning, Joey Huchette, and Miles Lubin. Jump: a modeling language for mathematical optimization. *SIAM Review*, 59(2):295–320, 2017. doi: 10.1137/15M1020575. URL <https://doi.org/10.1137/15M1020575>.
- [17] Lorenz T. Biegler and Ignacio E. Grossmann. Retrospective on optimization. *Computers & Chemical Engineering*, 28(8):1169–1192, 2004. ISSN 0098-1354. doi: <https://doi.org/10.1016/j.compchemeng.2003.11.003>. URL <https://www.sciencedirect.com/science/article/pii/S0098135403003089>.

- [18] NEOS Guide. Optimization problem types, 2025. URL <https://neos-guide.org/guide/types/>. Accessed: 2025-04-22.
- [19] Harshvardhan GM, Mahendra Kumar Gourisaria, Manjusha Pandey, and Siddharth Swarup Rautaray. A comprehensive survey and analysis of generative models in machine learning. *Computer Science Review*, 38:100285, 2020. doi: 10.1016/j.cosrev.2020.100285. URL <https://doi.org/10.1016/j.cosrev.2020.100285>.
- [20] Lynton Ardizzone, Jakob Kruse, Carsten Rother, and Ullrich Köthe. Analyzing inverse problems with invertible neural networks. In *International Conference on Learning Representations*, 2019. URL <https://openreview.net/forum?id=rJed6j0cKX>.
- [21] Agnimitra Dasgupta, Harisankar Ramaswamy, Javier Murgoitio-Esandi, Ken Y. Foo, Runze Li, Qifa Zhou, Brendan F. Kennedy, and Assad A. Oberai. Conditional score-based diffusion models for solving inverse elasticity problems. *Computer Methods in Applied Mechanics and Engineering*, 433:117425, 2025. doi: 10.1016/j.cma.2024.117425. URL <https://doi.org/10.1016/j.cma.2024.117425>.
- [22] John Kitchin. Solving an inverse problem with generative models, 2025. URL <https://doi.org/10.26434/chemrxiv-2025-nl9gl>.
- [23] Javier Alcazar, Mohammad Ghazi Vakili, Can B. Kalayci, and Alejandro Perdomo-Ortiz. Enhancing combinatorial optimization with classical and quantum generative models. *Nature Communications*, 15(1):2761, 2024. doi: 10.1038/s41467-024-46959-5. URL <https://doi.org/10.1038/s41467-024-46959-5>.
- [24] Michael S. Yao, Yimeng Zeng, Hamsa Bastani, Jacob Gardner, James C. Gee, and Osbert Bastani. Generative adversarial model-based optimization via source critic regularization. *arXiv*, page 2402.06532, 2024. doi: 10.48550/ARXIV.2402.06532. URL <https://arxiv.org/abs/2402.06532>.
- [25] Jonas Becker, Jan Philip Wahle, Bela Gipp, and Terry Ruas. Text generation: a systematic literature review of tasks, evaluation, and challenges. *arXiv*, page 2405.15604, 2024. doi: 10.48550/ARXIV.2405.15604. URL <https://arxiv.org/abs/2405.15604>.
- [26] Mohamed Elasri, Omar Elharrouss, Somaya Al-Maadeed, and Hamid Tairi. Image generation: a review. *Neural Processing Letters*, 54(5):4609–4646, 2022. doi: 10.1007/s11063-022-10777-x. URL <https://doi.org/10.1007/s11063-022-10777-x>.
- [27] Alexander Fabisch. Gmr: Gaussian mixture regression. *Journal of Open Source Software*, 6(62):3054, 2021. doi: 10.21105/joss.03054. URL <https://doi.org/10.21105/joss.03054>.
- [28] Alexia Jolicoeur-Martineau, Kilian Fatras, and Tal Kachman. Generating and imputing tabular data via diffusion and flow-based gradient-boosted trees. *arxiv*, page 2309.09968, 2023. doi: 10.48550/ARXIV.2309.09968. URL <https://arxiv.org/abs/2309.09968>.
- [29] D.H. Wolpert and W.G. Macready. No free lunch theorems for optimization. *IEEE Transactions on Evolutionary Computation*, 1(1):67–82, 1997. doi: 10.1109/4235.585893. URL <https://doi.org/10.1109/4235.585893>.
- [30] Linxiao Yang, Ngai-Man Cheung, Jiaying Li, and Jun Fang. Deep clustering by gaussian mixture variational autoencoders with graph embedding. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, October 2019.

- [31] Cinzia Viroli and Geoffrey J. McLachlan. Deep gaussian mixture models. *Statistics and Computing*, 29(1):43–51, 2019. ISSN 1573-1375. doi: 10.1007/s11222-017-9793-z. URL <https://doi.org/10.1007/s11222-017-9793-z>.
- [32] Christopher M Bishop and Nasser M Nasrabadi. *Pattern recognition and machine learning*, volume 4. Springer, 2006.
- [33] Alexander Fabisch. gmr: Gaussian mixture regression. *Journal of Open Source Software*, 6(62):3054, 2021. doi: 10.21105/joss.03054. URL <https://doi.org/10.21105/joss.03054>.
- [34] Marc Peter Deisenroth, A Aldo Faisal, and Cheng Soon Ong. *Mathematics for machine learning*. Cambridge University Press, 2020.
- [35] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.
- [36] Zoubin Ghahramani and Michael Jordan. Supervised learning from incomplete data via an em approach. In J. Cowan, G. Tesauro, and J. Alspector, editors, *Advances in Neural Information Processing Systems*, volume 6. Morgan-Kaufmann, 1993. URL https://proceedings.neurips.cc/paper_files/paper/1993/file/f2201f5191c4e92cc5af043eebfd0946-Paper.pdf.
- [37] J. Nocedal and S. Wright. *Numerical Optimization*. Springer Series in Operations Research and Financial Engineering. Springer New York, 2006. ISBN 9780387227429. URL <https://books.google.com/books?id=7wDpBwAAQBAJ>.
- [38] Victor Alves, John R. Kitchin, and Fernando V. Lima. An inverse mapping approach for process systems engineering using automatic differentiation and the implicit function theorem. *AIChE Journal*, page e18119, 2023. doi: 10.1002/aic.18119. URL <https://aiche.onlinelibrary.wiley.com/doi/abs/10.1002/aic.18119>.